RAPTOR: Resource-Aware Physics-Token Operator with Residual Local Adaptation for Aerodynamic Field Reconstruction

ARTICLE INFO

Keywords:
Aerodynamic field reconstruction
Physics-token neural operator
Sparse local adaptation
Computational scalability
Unstructured discretizations

ABSTRACT

Aerodynamic field reconstruction on unstructured discretizations requires long-range communication without losing local geometric detail, while dense node-level attention incurs quadratic cost. We propose RAPTOR, a resource-aware physics-token operator combining slice-based Physics-Attention with a sparse K-nearest-neighbor residual adapter. Its global branch communicates through physics tokens, while a zero-initialized bounded gate introduces node-space local correction without perturbing the backbone at initialization. For a fixed input state, each head's frozen node-mixing matrix has rank at most the number of slices S; with precomputed neighborhoods, RAPTOR retains $\mathcal{O}(NS + S^2 + NK)$ connectivity without dense node attention. We evaluate Reynolds-number and angle-of-attack extrapolation on AirfRANS and surface-pressure-coefficient reconstruction on Mixed-3D automobile and aircraft geometries. Across three training seeds on fixed AirfRANS splits, RAPTOR has the lowest mean macro relative L_2 error among ten models: 0.2210 ± 0.0019 for Reynolds and 0.3083 ± 0.0108 for angle of attack. It improves over its Transolver-Tiny backbone for all three common seeds, although the Reynolds margin is sensitive to one high-error baseline run. In single-seed Mixed-3D tests, RAPTOR reduces standardized-space MSE by 6.77% for automobiles and 8.57% for aircraft. A parameter-matched node-wise control and a neighborhood-size study support the contribution of local aggregation while exposing an accuracy–cost tradeoff. All accuracy experiments use 4096 nodes on one RTX 4050 Laptop GPU. In node-scaling profiling with precomputed neighborhoods, RAPTOR completes a mixed-precision training step at $N = 16384$ in 66.99 ms with 410.31 MiB peak allocated memory; dense attention is not run beyond $N = 8192$ to avoid memory oversubscription. The results support resource-efficient reconstruction on the tested hardware, while fixed splits and limited seeds constrain broader claims.

1. Introduction

High-fidelity computational fluid dynamics (CFD) is an essential tool for aerodynamic analysis and engineering design. Accurately solving the governing equations is often computationally expensive, particularly when repeated simulations are required for complex aerodynamic configurations such as airfoils, automobiles, and aircraft. Data-driven surrogate models provide an alternative by learning direct mappings from geometry and flow conditions to the corresponding physical fields, thereby reducing the cost of repeated CFD simulations (Bonnet et al., 2022; Elrefaie et al., 2024). Neural operators have emerged as an effective framework for learning mappings between function spaces (Lu et al., 2021; Li et al., 2021). However, aerodynamic field reconstruction on complex unstructured geometries remains challenging because it requires simultaneous modeling of long-range physical dependencies and local features such as surface curvature, boundary layers, leading-edge gradients, and wakes.

Existing architectures address these requirements from different perspectives. Point-cloud models such as PointNet aggregate global information through shared point encoders and symmetric pooling (Qi et al., 2017), whereas graph neural networks propagate node features through mesh edges or local neighborhood connections (Hamilton et al., 2017; Pfaff et al., 2021). These architectures introduce useful geometric inductive biases, but models relying exclusively on local propagation generally require multiple layers to establish cross-region communication. Global self-attention directly models non-local dependencies. However, applying dense attention to N nodes requires an $N \times N$ interaction matrix. For three-dimensional surface discretizations containing thousands or more nodes, this quadratic growth limits practical scalability.

Latent-token and physics-token operators reduce the cost of global interaction by projecting nodes into a compact latent space (Hao et al., 2023; Wu et al., 2024; Yang et al., 2026). Physics-Attention, in particular, softly assigns nodes to a limited number of physical slices, performs attention in the slice space, and maps the updated representations back to the original nodes (Wu et al., 2024). This mechanism enables efficient cross-region communication without dense node-level attention. But token compression may weaken local features associated with surface curvature, boundary layers, and high-gradient regions. A central challenge is therefore to recover such information without disrupting the global

operator or reintroducing quadratic complexity. This challenge is especially relevant under limited hardware budgets, since the high-memory GPUs required for larger token sets, wider representations, and extensive hyperparameter searches are not universally accessible. Developing more effective interaction structures for consumer-grade GPUs is thus important for improving the accessibility and reproducibility of aerodynamic field reconstruction.

To address this limitation, we propose RAPTOR, a residual-augmented physics-token operator for aerodynamic field reconstruction. RAPTOR retains slice-based Physics-Attention as its global backbone and equips each operator block with a sparse K-nearest-neighbor (K-NN) geometric adapter. The global branch captures cross-region dependencies through compressed slice interactions, whereas the local branch operates directly in node space to recover neighborhood information weakened by token compression. A zero-initialized bounded gate preserves the backbone at initialization and permits the local residual to be learned progressively. The main contributions of this work are summarized as follows:

1. **Node-space residual augmentation of a physics-token operator.** We propose a local correction mechanism that retains the original Physics-Attention pathway and augments each operator block with a parallel residual branch in the original node space. The branch concatenates the normalized center-node feature with the mean feature of its precomputed K-nearest neighbors and maps the result through a lightweight MLP. Unlike approaches that modify token assignment or inject geometric context into the latent slice space, RAPTOR directly corrects the evolving node representation while leaving the global tokenization and slice-attention mechanism unchanged.
2. **Backbone-preserving integration with controlled sparse overhead.** Building on the zero-initialized residual-scaling principle of ReZero, RAPTOR uses a bounded $\tanh(\alpha_l)$ gate to make the local branch contribute exactly zero at initialization. The resulting block therefore initially reproduces its Physics-Attention backbone and subsequently learns the magnitude and sign of the node-space correction. For fixed slice assignments and attention weights, we characterize the per-head conditional node-mixing rank of the token branch and show how the local adapter adds $\mathcal{O}(NK)$ direct neighborhood dependencies. With precomputed neighborhoods, the complete block retains $\mathcal{O}(NS + S^2 + NK)$ connectivity without constructing dense $N \times N$ node attention.
3. **Cross-dimensional evaluation and measured resource characterization.** We evaluate the same global–local design on two-dimensional AirfRANS extrapolation and three-dimensional automobile and aircraft surface-pressure reconstruction. Across three AirfRANS training seeds, RAPTOR obtains the lowest mean macro error among ten compared models on both extrapolation tasks; the three-dimensional experiments retain a single-seed protocol. A parameter-matched node-wise control and K-sensitivity study further test the role and cost of neighborhood information. We further profile forward latency, complete training-step latency, peak GPU memory, and exact CPU K-NN construction separately for up to $N = 16{,}384$ nodes on an RTX 4050 Laptop GPU, thereby distinguishing the cost of the sparse local correction from the one-time graph construction cost.

The remainder of this paper is organized as follows. Section 2 reviews related work on aerodynamic field reconstruction and neural operators. Section 3 introduces the RAPTOR architecture and analyzes its computational complexity. Section 4 characterizes conditional per-head node mixing and the baseline-preserving initialization. Section 5 describes the datasets, evaluation protocols, and resource-profiling procedure. Section 6 presents the quantitative comparisons, ablation studies, qualitative visualizations, and node-scaling results. Finally, Section 7 summarizes the findings, limitations, and future research directions.

2. Related Work

2.1. Aerodynamic Field Reconstruction and Geometric Learning

Data-driven aerodynamic field reconstruction aims to learn mappings from geometry and flow conditions to the corresponding physical fields, thereby reducing the cost of repeated numerical simulations. Machine learning has been applied to a broad range of fluid-mechanics problems, including flow-field prediction and flow control. Its effectiveness depends not only on model expressiveness, but also on the data distribution, incorporated physical priors, and evaluation methodology (Brunton et al., 2020). Early studies mainly employed convolutional encoder–decoder architectures to predict velocity and pressure distributions around airfoils. Thuerey et al. (2020) investigated how network capacity and training-set size affect the prediction accuracy of Reynolds-averaged Navier–Stokes (RANS) flow fields. Bhatnagar et al. (2019) further examined predictive capability across different airfoils, angles of attack, and Reynolds numbers. Public datasets such as AirfRANS and DrivAerNet++ have subsequently enabled reproducible

comparisons for two-dimensional airfoil and three-dimensional automotive problems (Bonnet et al., 2022; Elrefaie et al., 2024).

For unstructured discretizations, point-cloud and graph-based models can directly process nodes and their connectivity. PointNet obtains a permutation-invariant global shape representation through a shared point encoder and symmetric aggregation (Qi et al., 2017). GraphSAGE performs local information propagation through neighborhood sampling and feature aggregation (Hamilton et al., 2017), whereas Graph U-Nets construct hierarchical representations using graph pooling, unpooling, and skip connections (Gao and Ji, 2019). MeshGraphNets further extend graph message passing to mesh-based physical simulations (Pfaff et al., 2021). More recently, Immordino et al. (2025) employed an autoencoder graph convolutional network to predict transonic flow fields on non-homogeneous unstructured grids. Unlike architectures that rely predominantly on neighborhood propagation, RAPTOR establishes a global pathway through physics-aware slices and restricts neighborhood propagation to a local residual correction.

2.2. Neural Operators and Physics-Token Interactions on Complex Geometries

Neural operators extend the learning objective from finite-dimensional vector mappings to mappings between function spaces. Their theoretical foundation can be traced to approximation results for continuous nonlinear operators (Chen and Chen, 1995). DeepONet combines branch and trunk networks (Lu et al., 2021), whereas the Fourier Neural Operator parameterizes nonlocal interactions in the frequency domain (Li et al., 2021). Kovachki et al. (2023) presented a unified formulation covering integral-kernel, low-rank, graph-based, and Fourier representations; related reviews summarize their use in scientific simulation and engineering design (Azizzadenesheli et al., 2024).

Early neural operators commonly assumed regular grids or fixed parameterized domains. Geo-FNO learns a deformation to a regular latent domain (Li et al., 2023); DIMON uses diffeomorphic mappings between reference and target domains (Yin et al., 2024); and PCNO constructs operators directly on irregular point clouds (Zeng et al., 2025). GINOT combines a geometric encoder with an attention-based query decoder (Liu et al., 2026), while GAOT uses multiscale graph encoding, geometric embeddings, and Transformer processors (Wen et al., 2025). These studies show that geometric representation directly affects generalization across shapes.

Another class of methods organizes global information using a finite number of latent tokens. GNOT applies a neural operator Transformer to irregular discretizations (Hao et al., 2023). Transolver introduces Physics-Attention, which aggregates nodes into physics-aware slices, performs interactions in slice space, and maps the updated features back to nodes (Wu et al., 2024). Transolver++ refines token assignment through point-dependent temperatures and slice reparameterization (Luo et al., 2025). RAPTOR addresses a different locality: it leaves the slice mechanism intact and adds an explicit K-NN residual correction in node space. LinearNO reformulates Physics-Attention through asymmetric linear attention (Hu et al., 2026), while MoNo uses entropy-regularized optimal transport to balance hierarchical latent assignments (Yang et al., 2026).

Unlike studies that redesign global slice interactions or multiscale token assignments, RAPTOR preserves the Physics-Attention backbone and investigates how a limited amount of node-level local computation can enhance its representation.

2.3. Local–Global Complementarity and Residual Adaptation

Modeling global dependencies in a compressed space can reduce interaction costs, but local structures may be weakened. LOCA uses coupled attention (Kissas et al., 2022), Latent DeepONet learns a low-dimensional representation (Kontolati et al., 2024), and U-FNO combines a U-Net with a Fourier operator (Wen et al., 2022). GAOT similarly combines geometry-aware representations with Transformer interactions (Wen et al., 2025). These designs motivate examining whether explicit neighborhood computation can complement compressed global interaction.

Local dependence has also become an explicit subject of neural-operator research. Chen et al. (2025) constrained the information available to local predictions through data decomposition and studied its effects on training and generalization. Architectures that rely entirely on local propagation require greater depth for cross-regional interaction, whereas a randomly initialized local branch can perturb an established global backbone early in training.

ReZero initializes residual coefficients to zero and progressively activates residual transformations during optimization (Bachlechner et al., 2021). RAPTOR applies this principle to a bounded node-level adapter gate, preserving the function represented by an existing global operator at initialization rather than requiring the entire block to reduce to an identity map.

GeoTransolver is closely related in its objective of preserving geometric information within a Physics-Attention architecture (Adams et al., 2025). However, the two methods inject information at different representation levels.

GeoTransolver constructs a shared geometry and operating-condition context using multi-scale ball queries and repeatedly injects this context into the slice-token representation through cross-attention. RAPTOR instead leaves the Physics-Attention token pathway unchanged and introduces a parallel correction directly in the evolving node representation. Its neighborhoods are defined by a single precomputed directed Euclidean K-NN graph among the sampled field nodes, and its local message is formed from the center-node feature and the mean feature of its neighbors. Consequently, GeoTransolver targets persistent multiscale geometry and regime conditioning in token space, whereas RAPTOR targets a lightweight node-space correction of neighborhood information that may be weakened by token compression. The methods are therefore complementary rather than architecturally equivalent.

2.4. Implementation under a Fixed Hardware Budget

The computational efficiency of neural operators is commonly evaluated using asymptotic complexity, parameter count, throughput, or parallel scalability (Azizzadenesheli et al., 2024). Transolver++ targets million-point inputs and multi-GPU scaling (Luo et al., 2025); LinearNO simplifies Physics-Attention (Hu et al., 2026); and GAOT reports throughput and latency on high-performance GPUs (Wen et al., 2025).

However, lower asymptotic complexity does not necessarily translate into shorter runtime or lower peak memory. Operator fusion, intermediate activations, neighborhood indexing, and hardware parallelism all affect measured cost. This study therefore reports parameters, peak allocated memory, and elapsed training time for the formal $N = 4096$ experiments, and supplements them with an isolated node-scaling profile over $N \in \{1024, 2048, 4096, 8192, 16384\}$. The profiling experiment measures forward-pass latency, full mixed-precision training-step latency, and peak allocated memory on the laptop GPU, and separately measures exact K-NN graph-construction time on the host CPU.

3. RAPTOR Method

RAPTOR is designed to reconstruct aerodynamic fields on irregularly discretized geometries while avoiding dense node-to-node attention. As illustrated in Figure 1, the model combines a token-mediated Physics-Attention backbone with a sparse local geometric adapter. The former establishes long-range physical communication through a compressed token space, whereas the latter restores neighborhood-scale information that may be weakened during token compression. A zero-initialized bounded gate controls the contribution of the local branch.

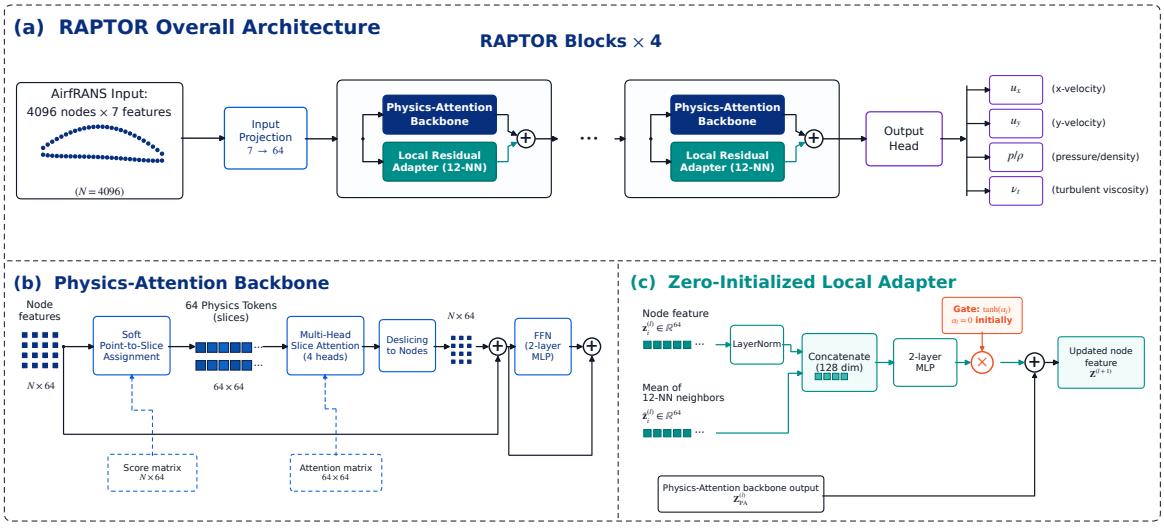


Figure 1: Architecture of RAPTOR. (a) Overall prediction architecture illustrated using the AirfRANS configuration. (b) Multi-head Physics-Attention backbone, which compresses node features into physics tokens, performs global token interaction, and maps the updated tokens back to the original nodes. (c) Zero-initialized local adapter. LayerNorm is applied only to the center-node feature before it is concatenated with the unnormalized mean of its K neighbors; the resulting local message is gated and added to the Physics-Attention backbone output.

3.1. Overall architecture

Let an aerodynamic sample be represented by N irregularly distributed nodes with input attributes $\mathbf{X} = [\mathbf{x}_1, \dots, \mathbf{x}_N]^\top \in \mathbb{R}^{N \times d_{\text{in}}}$, where d_{in} depends on the task. The raw node attributes may contain spatial coordinates, geometric descriptors, boundary information, and flow conditions. An input projection maps these heterogeneous variables into a shared d-dimensional latent space:

$$\mathbf{Z}^{(0)} = \phi_{\text{in}}(\mathbf{X}) \in \mathbb{R}^{N \times d}. \tag{1}$$

The latent width is set to $d = 64$ in the experiments. This width provides sufficient capacity for cross-variable representation while remaining compatible with the memory constraints of the target consumer-grade GPU. The latent representation is processed by L stacked RAPTOR blocks:

$$\mathbf{Z}^{(l+1)} = \mathcal{R}^{(l)}\left(\mathbf{Z}^{(l)}, \mathcal{N}_K\right), \qquad l = 0, \dots, L-1, \tag{2}$$

where $\mathcal{N}_K$ denotes a fixed Euclidean K-nearest-neighbor graph constructed from the node coordinates. Self-neighbors are excluded; the queried neighborhoods are directed and are not symmetrized. The indices are precomputed once for every sampled geometry and reused throughout training and evaluation. Each block contains a Physics-Attention global backbone and a gated local residual adapter. The final aerodynamic field is decoded as:

$$\hat{\mathbf{Y}} = \phi_{\text{out}}\left(\mathbf{Z}^{(L)}\right) \in \mathbb{R}^{N \times d_{\text{out}}}. \tag{3}$$

The input and output dimensions are task-dependent. For the AirfRANS experiments shown in Fig. 1, $N = 4096$, $d_{\text{in}} = 7$, and $d_{\text{out}} = 4$. The seven inputs are the two node coordinates, two inlet-velocity components, distance to the airfoil, and two surface-normal components. The four outputs are u_x, u_y, pressure divided by density p/ρ, and turbulent kinematic viscosity ν_t. For the three-dimensional automobile and aircraft experiments, the input is the raw three-dimensional surface coordinate and the output is the surface pressure coefficient C_p. The input projection is linear. The output head applies LayerNorm, a linear layer, GELU, and a final linear layer.

3.2. Multi-head Physics-Attention backbone

Let $\mathbf{Z}^{(l)} \in \mathbb{R}^{N \times d}$ denote the input to the l-th RAPTOR block. The Physics-Attention branch first applies LayerNorm and projects the normalized representation into feature and slice-assignment spaces:

$$\mathbf{U}^{(l)} = \text{LN}_1^{(l)}\left(\mathbf{Z}^{(l)}\right). \tag{4}$$

The latent dimension is divided among H attention heads, with $d_h = d/H$. For head h, the projected node and value features are:

$$\mathbf{X}_h^{(l)} = P_{x,h}^{(l)}\left(\mathbf{U}^{(l)}\right), \qquad \mathbf{V}_h^{(l)} = P_{f,h}^{(l)}\left(\mathbf{U}^{(l)}\right), \tag{5}$$

where $\mathbf{X}_h^{(l)}, \mathbf{V}_h^{(l)} \in \mathbb{R}^{N \times d_h}$.

Each attention head learns its own soft point-to-slice assignment:

$$\mathbf{W}_h^{(l)} = \text{Softmax}_S\left(\frac{\mathbf{X}_h^{(l)} \mathbf{W}_{s,h}^{(l)}}{\tau_h^{(l)}}\right) \in \mathbb{R}^{N \times S}, \tag{6}$$

where S is the number of physics slices and $\tau_h^{(l)}$ is a learnable temperature initialized to 0.5 and clamped to $[0.1, 5.0]$. The softmax is applied over the S slices, so that: $w_{h,is}^{(l)} \geq 0, \qquad \sum_{s=1}^{S} w_{h,is}^{(l)} = 1.$

The mass of each physics slice is:

$$d_{h,s}^{(l)} = \sum_{i=1}^{N} w_{h,is}^{(l)}, \qquad \mathbf{D}_h^{(l)} = \text{diag}\left(d_{h,1}^{(l)}, \dots, d_{h,S}^{(l)}\right). \tag{7}$$

The node features are compressed into S physics tokens using mass-normalized aggregation:

$$\mathbf{T}_h^{(l)} = \left(\mathbf{D}_h^{(l)} + \varepsilon \mathbf{I}\right)^{-1} \left(\mathbf{W}_h^{(l)}\right)^\top \mathbf{V}_h^{(l)} \in \mathbb{R}^{S \times d_h}, \tag{8}$$

where $\varepsilon > 0$ prevents numerical division by zero.

Queries, keys, and values are computed in the token space:

$$\mathbf{Q}_h^{(l)} = \mathbf{T}_h^{(l)}\mathbf{W}_{Q,h}^{(l)}, \quad \mathbf{K}_h^{(l)} = \mathbf{T}_h^{(l)}\mathbf{W}_{K,h}^{(l)}, \quad \mathbf{V}_{T,h}^{(l)} = \mathbf{T}_h^{(l)}\mathbf{W}_{V,h}^{(l)}. \tag{9}$$

The slice-level attention matrix and updated tokens are:

$$\mathbf{A}_h^{(l)} = \mathrm{Softmax}\left(\frac{\mathbf{Q}_h^{(l)}\left(\mathbf{K}_h^{(l)}\right)^{\top}}{\sqrt{d_h}}\right) \in \mathbb{R}^{S\times S}, \tag{10}$$

$$\widetilde{\mathbf{T}}_h^{(l)} = \mathbf{A}_h^{(l)}\mathbf{V}_{T,h}^{(l)}. \tag{11}$$

The updated tokens are mapped back to the original nodes using the same soft assignment:

$$\mathbf{G}_h^{(l)} = \mathbf{W}_h^{(l)}\widetilde{\mathbf{T}}_h^{(l)} \in \mathbb{R}^{N\times d_h}. \tag{12}$$

Outputs from all attention heads are concatenated and linearly projected:

$$\mathbf{G}^{(l)} = P_o^{(l)}\left(\mathrm{Concat}\left[\mathbf{G}_1^{(l)}, \ldots, \mathbf{G}_H^{(l)}\right]\right). \tag{13}$$

The Physics-Attention backbone contains two residual updates. Its feed-forward network uses expansion ratio four, $d \to 4d \to d$:

$$\mathbf{U}_{\mathrm{PA}}^{(l)} = \mathbf{Z}^{(l)} + \mathbf{G}^{(l)}, \tag{14}$$

$$\mathbf{Z}_{\mathrm{PA}}^{(l)} = \mathbf{U}_{\mathrm{PA}}^{(l)} + \mathrm{FFN}^{(l)}\left(\mathrm{LN}_2^{(l)}\left(\mathbf{U}_{\mathrm{PA}}^{(l)}\right)\right). \tag{15}$$

Global communication is therefore performed among S physics tokens instead of directly among all N nodes. This construction avoids an explicit dense $N \times N$ node-attention matrix while retaining a global information pathway.

3.3. Zero-initialized local residual adapter

Although token-mediated interaction provides an efficient global pathway, compressing N nodes into S physics tokens can weaken information associated with local surface continuity, geometric curvature, boundary layers, and sharp field variations. RAPTOR therefore introduces a sparse local adapter operating directly in the original node space. For node i, let $\mathcal{N}_K(i)$ denote the indices of its K nearest neighbors. The neighborhood feature is computed by mean aggregation: $\bar{\mathbf{z}}_i^{(l)} = \frac{1}{K}\sum_{j\in\mathcal{N}_K(i)}\mathbf{z}_j^{(l)}$.

Consistent with the implementation, LayerNorm is applied to the center-node feature before concatenation, while the aggregated neighborhood feature is used directly. The local message is:

$$\mathbf{m}_i^{(l)} = \phi_{\mathrm{loc}}^{(l)}\left(\left[\mathrm{LN}_{\mathrm{loc}}^{(l)}\left(\mathbf{z}_i^{(l)}\right); \bar{\mathbf{z}}_i^{(l)}\right]\right), \tag{16}$$

where $[\cdot;\cdot]$ denotes feature concatenation and $\phi_{\mathrm{loc}}^{(l)}$ is a two-layer MLP with GELU activation and hidden width d: $2d \to d \to d$.

Collecting the node-level messages gives: $\mathbf{M}_{\mathrm{loc}}^{(l)} = \left[\mathbf{m}_1^{(l)}, \ldots, \mathbf{m}_N^{(l)}\right]^{\top} \in \mathbb{R}^{N\times d}$. The local message is modulated by a learnable scalar gate: $\gamma_l = \tanh(\alpha_l)$, $\quad -1 < \gamma_l < 1$, and the output of the l-th RAPTOR block is:

$$\mathbf{Z}^{(l+1)} = \mathbf{Z}_{\mathrm{PA}}^{(l)} + \tanh(\alpha_l)\mathbf{M}_{\mathrm{loc}}^{(l)}. \tag{17}$$

The local adapter takes the original block input $\mathbf{Z}^{(l)}$, whereas $\mathbf{Z}_{\mathrm{PA}}^{(l)}$ is obtained after the Physics-Attention and feed-forward residual updates. This parallel construction preserves the original global backbone and adds an independent local correction. Each gate is initialized as $\alpha_l = 0$. Consequently, the local branch contributes exactly zero at initialization. Its effect is activated progressively during optimization, as analyzed in Section 4.

3.4. Training objective and reconstruction procedure

Let $\widehat{\mathbf{Y}}^{(q)}$ and $\mathbf{Y}^{(q)}$ denote the normalized prediction and ground-truth field of training sample q. The model is trained by minimizing the node-averaged mean-squared error:

$$\mathcal{L}_{\text{train}} = \frac{1}{N_{\text{tr}}} \sum_{q=1}^{N_{\text{tr}}} \frac{1}{N_q d_{\text{out}}} \left\| \widehat{\mathbf{Y}}^{(q)} - \mathbf{Y}^{(q)} \right\|_F^2 . \tag{18}$$

Normalization statistics are computed using only the training set. For AirfRANS, all seven inputs and four outputs are standardized channel-wise. For the three-dimensional tasks, coordinates remain in their stored units and only C_p is standardized using the scalar training-set mean and standard deviation. Model selection uses validation loss with early stopping; the validation-selected checkpoint is restored for test evaluation, after which predictions are transformed back to physical units where required. Because the surrounding optimizer and early-stopping loop is standard, Algorithm 1 separates the RAPTOR-specific setup from the repeatedly executed forward pass. InitializeRAPTOR is called once before optimization, CacheKNNGraph is called once for each sampled geometry, and RAPTORForward is called repeatedly during training and inference without reinitializing any parameter.

Algorithm 1 Separated RAPTOR initialization, graph caching, and forward propagation

Input: Sampled node features $\mathbf{X}$, coordinates $\mathbf{C} = \{\mathbf{c}_i\}_{i=1}^{N}$, fitted input preprocessor $\mathcal{P}_X$, slices S, neighbors K, and blocks L

procedure InitializeRAPTOR(S, L)
  Initialize all non-gate trainable parameters in θ
  Set $\alpha_l \leftarrow 0$ for $l = 0, \dots, L-1$
  return initialized model parameters θ
end procedure
procedure CacheKNNGraph($\mathbf{C}, K$)
  for all nodes $i = 1, \dots, N$ **do**
    $\mathcal{N}_K(i) \leftarrow \mathrm{KNN}_K(\mathbf{c}_i, \{\mathbf{c}_j : j \neq i\})$
  end for
  Cache and **return** the directed, self-excluding graph $\mathcal{N}_K$
end procedure
procedure RAPTORForward($\mathbf{X}, \mathcal{N}_K; \theta$)
  $\mathbf{Z}^{(0)} \leftarrow \phi_{\text{in}}\big(\mathcal{P}_X(\mathbf{X})\big)$
  for $l = 0, \dots, L-1$ **do**
    $\mathbf{U}_{\text{PA}}^{(l)} \leftarrow \mathbf{Z}^{(l)} + \mathrm{PhysicsAttn}^{(l)}\Big(\mathrm{LN}_1^{(l)}(\mathbf{Z}^{(l)}), S\Big)$
    $\mathbf{Z}_{\text{PA}}^{(l)} \leftarrow \mathbf{U}_{\text{PA}}^{(l)} + \mathrm{FFN}^{(l)}\Big(\mathrm{LN}_2^{(l)}(\mathbf{U}_{\text{PA}}^{(l)})\Big)$
    for all nodes $i = 1, \dots, N$ in parallel **do**
      $\bar{\mathbf{z}}_i^{(l)} \leftarrow K^{-1} \sum_{j \in \mathcal{N}_K(i)} \mathbf{z}_j^{(l)}$
      $\mathbf{m}_i^{(l)} \leftarrow \phi_{\text{loc}}^{(l)}\Big([\mathrm{LN}_{\text{loc}}^{(l)}(\mathbf{z}_i^{(l)}); \bar{\mathbf{z}}_i^{(l)}]\Big)$
    end for
    $\mathbf{M}_{\text{loc}}^{(l)} \leftarrow [\mathbf{m}_1^{(l)}, \dots, \mathbf{m}_N^{(l)}]^\top$
    $\mathbf{Z}^{(l+1)} \leftarrow \mathbf{Z}_{\text{PA}}^{(l)} + \tanh(\alpha_l)\mathbf{M}_{\text{loc}}^{(l)}$
  end for
  $\widehat{\mathbf{Y}} \leftarrow \phi_{\text{out}}(\mathbf{Z}^{(L)})$
  return normalized field prediction $\widehat{\mathbf{Y}}$
end procedure

3.5. Connectivity and computational complexity

Dense node-level self-attention constructs interactions between every pair of nodes. Its logical attention-score count is $|\mathcal{E}_{\text{dense}}| = \mathcal{O}(HN^2)$, and principal interaction complexity is $C_{\text{dense}} = \mathcal{O}(N^2 d)$, where H is the number of attention heads. For RAPTOR, every attention head constructs NS point-to-slice scores and S^2 slice-to-slice attention scores. The local adapter introduces NK directed neighborhood dependencies. Therefore, one RAPTOR block has the logical interaction count $|\mathcal{E}_{\text{RAPTOR}}| = \mathcal{O}\left(HNS + HS^2 + NK\right)$. When H is treated as a fixed architectural constant, this reduces to $|\mathcal{E}_{\text{RAPTOR}}| = \mathcal{O}\left(NS + S^2 + NK\right)$. The interaction-related arithmetic complexity is $C_{\text{interaction}} = \mathcal{O}\left(NSd + S^2 d + NKd\right)$. Feature projections, the feed-forward network, and the two-layer local MLP additionally introduce an $\mathcal{O}(Nd^2)$ term. The complete per-block complexity is therefore $C_{\text{block}} = \mathcal{O}\left(NSd + S^2 d + NKd + Nd^2\right)$. For L blocks, the total complexity is $C_{\text{RAPTOR}} = \mathcal{O}\left[L\left(NSd + S^2 d + NKd + Nd^2\right)\right]$. When S, K, H, d, and L

remain fixed, both the interaction count and arithmetic complexity grow linearly with N. This statement excludes graph construction. Exact Euclidean K-NN search costs $\mathcal{O}(N^2)$ with the pairwise implementation used for the three-dimensional caches (or approximately $\mathcal{O}(N \log N)$ with the k-d tree used for AirfRANS), plus $\mathcal{O}(NK)$ storage. Because graphs are precomputed, these costs are not included in the reported training time or peak allocated training memory.

Euclidean proximity is only an approximation to surface adjacency. A directed neighbor may cross a thin wing, a vehicle gap, or another nearby but geodesically distant surface. The present experiments do not prevent such connections; mesh-edge or geodesic neighborhoods are therefore an important control for future work.

4. Theoretical Properties of RAPTOR

4.1. Conditional global mixing and sparse local support

A RAPTOR block can be expressed as:

$$\mathcal{R}_{\text{RAPTOR}}\left(\mathbf{Z}, \mathcal{N}_K\right) = \mathcal{P}_{\text{global}}(\mathbf{Z}) + \gamma_\alpha \mathcal{A}_{\text{local}}\left(\mathbf{Z}, \mathcal{N}_K\right), \qquad \gamma_\alpha = \tanh(\alpha), \tag{19}$$

where $\mathcal{P}_{\text{global}}$ is the complete Physics-Attention backbone, including its attention and feed-forward residual updates, and $\mathcal{A}_{\text{local}}$ is the sparse K-NN adapter.

Equation (19) is an architectural decomposition, not a low-rank decomposition of the complete nonlinear operator or its Jacobian. Both the point-to-slice assignment and the slice-level attention matrix depend on $\mathbf{Z}$; the local branch also contains normalization, GELU, and a nonlinear MLP.

For a fixed input state, however, the node-mixing pattern of each Physics-Attention head is mediated by only S physics slices, whereas the direct dependencies of the local branch are supported on the sparse K-NN graph. RAPTOR can therefore be interpreted structurally as:

$$\underbrace{\text{token-mediated global interaction}}_{\text{low-rank per attention head}} + \underbrace{K\text{-NN local correction}}_{\text{sparse node-level support}}. \tag{20}$$

The two branches serve complementary purposes. The global branch provides communication between distant aerodynamic regions, whereas the local branch preserves neighborhood-scale geometric and physical variation.

4.2. Conditional rank of a Physics-Attention head

Consider one attention head h at a fixed input state and omit the block superscript for clarity. Let $\mathbf{W}_h \in \mathbb{R}^{N\times S}$ denote its node-to-slice assignment and $\mathbf{D}_h = \operatorname{diag}\left(\mathbf{W}_h^\top \mathbf{1}_N\right)$ slice-mass matrix. Let $\mathbf{A}_h \in \mathbb{R}^{S\times S}$ denote the corresponding slice-attention matrix. Ignoring feature projections, which act on the channel dimension rather than directly increasing the number of node-mixing modes, the effective node-space interaction of head h is:

$$\mathbf{K}_h = \mathbf{W}_h \mathbf{A}_h \left(\mathbf{D}_h + \varepsilon \mathbf{I}\right)^{-1} \mathbf{W}_h^\top \in \mathbb{R}^{N\times N}. \tag{21}$$

Proposition 4.1 (Per-head frozen conditional rank)**.** *For each Physics-Attention head h, after freezing the input-dependent assignment and slice-attention matrices, the effective node-level interaction matrix in Equation (21) satisfies* $\operatorname{rank}(\mathbf{K}_h) \le S$.

Proof. The matrix $\mathbf{K}_h$ is a product whose node-to-slice factors have inner dimension S. Therefore,

$$\operatorname{rank}(\mathbf{K}_h) \le \operatorname{rank}(\mathbf{W}_h) \le S. \tag{22}$$

□

Proposition 4.1 formalizes only a frozen, per-head node-mixing statement. It is not a rank bound for the complete multi-head nonlinear block or its Jacobian, because the assignments, attention matrices, concatenation, and channel-wise output projection are input dependent.

4.3. Sparse support of the local correction

The local adapter produces the node-level message $\mathbf{m}_i = \phi_{\text{loc}}\left(\left[\text{LN}(\mathbf{z}_i); \frac{1}{K}\sum_{j\in\mathcal{N}_K(i)}\mathbf{z}_j\right]\right)$. For a fixed neighborhood graph, $\mathbf{m}_i$ depends directly only on the center node i and the nodes in $\mathcal{N}_K(i)$. Consequently, the direct node-dependency graph contains at most $N(K+1)$ center and neighbor relations: $|\mathcal{E}_{\text{local}}| = \mathcal{O}(NK)$.

Because ϕ_{loc} is nonlinear, the local adapter is not a fixed sparse linear matrix. Its sparsity refers to the support of its direct node dependencies, or equivalently to the sparsity pattern of its node-wise Jacobian for a fixed K-NN graph. This sparse support complements the token-mediated global interaction. It introduces direct geometric communication without restoring dense node-level attention.

4.4. Baseline recovery and progressive adapter activation

Let

$$\mathbf{Z}^+ = \mathbf{Z}_{\text{PA}} + \tanh(\alpha)\mathbf{M}_{\text{loc}} \tag{23}$$

denote the output of one RAPTOR block.

Proposition 4.2 (Exact backbone recovery at initialization)**.** *If the local gate is initialized as $\alpha_0 = 0$, then the RAPTOR block is exactly equivalent to its Physics-Attention backbone at initialization: $\mathbf{Z}^+\big|_{\alpha=0} = \mathbf{Z}_{\text{PA}}$.*

Proof. Since $\tanh(0) = 0$, Equation (23) gives $\mathbf{Z}^+\big|_{\alpha=0} = \mathbf{Z}_{\text{PA}} + 0 \cdot \mathbf{M}_{\text{loc}} = \mathbf{Z}_{\text{PA}}$. □

The equality in Proposition 4.2 is exact rather than asymptotic. If the Physics-Attention and feed-forward parameters are copied from a backbone model, RAPTOR produces identical initial outputs, up to numerical precision.

Although the local message does not contribute to the initial forward pass, the gate can still receive a nonzero gradient. For a scalar loss $\mathcal{L}$,

$$\frac{\partial \mathcal{L}}{\partial \alpha} = \left\langle \frac{\partial \mathcal{L}}{\partial \mathbf{Z}^+}, \mathbf{M}_{\text{loc}} \right\rangle_F \left(1 - \tanh^2(\alpha)\right), \tag{24}$$

where $\langle\cdot,\cdot\rangle_F$ denotes the Frobenius inner product. At initialization,

$$\left.\frac{\partial \mathcal{L}}{\partial \alpha}\right|_{\alpha=0} = \left\langle \frac{\partial \mathcal{L}}{\partial \mathbf{Z}^+}, \mathbf{M}_{\text{loc}} \right\rangle_F, \tag{25}$$

which is generally nonzero.

By contrast, if ϑ_{loc} denotes a parameter inside the local MLP, then $\left.\frac{\partial \mathcal{L}}{\partial \vartheta_{\text{loc}}}\right|_{\alpha=0} = 0$, because the local message is multiplied by $\tanh(0) = 0$.

This property prevents the randomly initialized local MLP from immediately perturbing the Physics-Attention representation while still allowing the optimizer to determine whether and in which direction the local correction should be activated.

Remark 4.1 (Bounded scalar multiplier)**.** *Because* $|\tanh(\alpha)| \le 1$,

$$\left\|\mathbf{Z}^+ - \mathbf{Z}_{\text{PA}}\right\|_F \le \left\|\mathbf{M}_{\text{loc}}\right\|_F.$$

This direct norm inequality bounds only the scalar multiplier; it does not bound the nonlinear local message independently of the input.

Remark 4.2 (Conditional branch complementarity)**.** *Suppose that an aerodynamic field contains long-range dependencies that can be represented effectively by token-mediated global interaction and a remaining component that is predictable from local geometric neighborhoods. Under this condition, the Physics-Attention backbone and the K-NN adapter provide complementary approximation mechanisms. This interpretation explains the motivation for RAPTOR but does not constitute a universal superiority theorem. The actual benefit of the local branch must be established empirically through matched-baseline comparisons and ablation experiments.*

Table 1
Locally downloaded aerodynamic datasets. Sizes are uncompressed and reported in decimal GB.

Dataset	Samples	Task or geometry family	Size (GB)	Source
AirfRANS	1,000	Reynolds and AoA extrapolation	15.03	AirfRANS
Cars	5,428	Estateback, Fastback, Notchback	4.81	Mixed-3D; automobiles derived from DrivAerNet++
Aircraft	6,766	J20, P180, Boeing 737, ERJ	6.13	Mixed-3D benchmark

5. Experimental Setup

RAPTOR is evaluated through a progression from two-dimensional airfoils to three-dimensional automobiles and aircraft. The AirfRANS tasks assess field reconstruction under unseen flow conditions; the automobile task examines reconstruction on complex three-dimensional surfaces; and the aircraft task tests applicability across distinct aerodynamic geometries. An overview of the locally downloaded datasets is given in Table 1.

5.1. Datasets

(1) Two-dimensional airfoils. The AirfRANS dataset (Bonnet et al., 2022) contains steady two-dimensional Reynolds-averaged Navier–Stokes simulations over varying airfoil geometries, Reynolds numbers, and angles of attack. We evaluate Reynolds-number and angle-of-attack extrapolation using the official task-specific partitions. For each task, 200 simulations are selected from the official training partition with selection seed 2026 and split into 180 training and 20 validation cases. The official test partitions provide 200 Reynolds-number and 196 AoA cases for this evaluation. Each case is downsampled to $N = 4096$ nodes using 20% surface and 80% volume samples. Sampling uses deterministic split offsets 1000, 5000, and 9000 for training, validation, and test cases, respectively. This is a custom downsampled evaluation rather than the official full-node metric. The seven inputs are x, y, the two inlet-velocity components, distance to the airfoil, and the two normal components. Targets are u_x, u_y, pressure divided by density p/ρ, and turbulent kinematic viscosity ν_t.

(2) Three-dimensional automobiles. The automobile task uses the Mixed-3D benchmark (Zeng et al., 2025; Zhong et al., 2025), whose automobile geometries are derived from DrivAerNet++ (Elrefaie et al., 2024). Mixed-3D provides surface pressure coefficients C_p computed by a panel method under the prescribed freestream $\mathbf{v}_\infty = [1, 0, 0]$. We select 100 vehicles with seed 42, partition them into 80/10/10 training, validation, and test objects, and sample 4096 surface nodes per object. The model receives three-dimensional coordinates and predicts C_p.

(3) Three-dimensional aircraft. The aircraft task uses the OpenVSP-derived portion of Mixed-3D (Zeng et al., 2025; Zhong et al., 2025) and the same panel-method C_p target. We select 25 objects from each of the J20, P180, Boeing 737, and ERJ families with seed 42. Each family contributes 20 training objects, while the stratified validation and test allocations sum to 10 objects each. Every surface is sampled to 4096 nodes. The complete automobile and aircraft object identifiers are supplied in the accompanying split-manifest files.

5.2. Comparison baselines

The AirfRANS comparison includes an independent-node MLP; PointNet, GraphSAGE, and Graph U-Net geometric baselines (Qi et al., 2017; Hamilton et al., 2017; Gao and Ji, 2019); a controlled Sparse Token implementation inspired by SEA (Lee et al., 2024); and four compact operator baselines. All "Tiny" operators use the common width, depth, and latent-state budget in Table 2. Transolver-Tiny implements Physics-Attention and is RAPTOR's direct no-adapter backbone baseline (Wu et al., 2024). Transolver++-Tiny retains point-dependent temperature and slice reparameterization (Luo et al., 2025); LinearNO-Tiny uses asymmetric linear attention (Hu et al., 2026); and MoNo-Tiny retains hierarchical latent modes with balanced assignment (Yang et al., 2026). These are resource-matched implementations rather than the full-scale configurations of the original studies.

The automobile experiment uses PointNet and Transolver-Tiny as primary baselines. The aircraft evaluation compares RAPTOR with Transolver-Tiny under the same stratified split. Because both models are retrained and all four families occur in training and test sets, this experiment assesses applicability across aircraft geometries rather than transfer learning or unseen-family extrapolation.

5.3. Unified experimental protocol

All formal training and evaluation runs are conducted on a Windows-based laptop equipped with an Intel Core Ultra 9 185H CPU, 32 GB of system memory (31.5 GB usable), and a single NVIDIA GeForce RTX 4050 Laptop GPU with 6141 MiB of device memory. The recorded NVIDIA driver version is 596.58. The software environment uses Python 3.12.13, PyTorch 2.5.1, CUDA 12.1, NumPy 2.3.1, and SciPy 1.16.0. The unified hyperparameter configuration is summarized in Table 2.

CUDA automatic mixed precision is enabled for the AirfRANS and three-dimensional runs. The mixed-feature scatter in MoNo-Tiny and the eidetic slicing operation in Transolver++-Tiny are explicitly evaluated in FP32 because their corresponding kernels were not autocast safe. Seed 42 is the reference run for all tasks and for the detailed AirfRANS comparisons in Tables 3 and 4. The AirfRANS model comparison is repeated with training seeds 41 and 43, holding the data caches, sampled nodes, normalization statistics, hyperparameters, and stopping rule fixed. The three-dimensional experiments, the capacity and K studies, and the remaining ablations retain the seed-42 protocol. The two three-dimensional split manifests record every object identifier and the sampling seed.

Node-scaling profiling protocol. Computational scaling is evaluated without retraining by applying the actual Transolver-Tiny and RAPTOR implementations to deterministic synthetic tensors with three input channels and one output channel. An explicit dense node-attention baseline materializes the per-head $N \times N$ attention matrix. The node count is varied over $N \in \{1024, 2048, 4096, 8192, 16384\}$, while the number of physics slices $S = 64$, neighborhood size $K = 12$, hidden width $d = 64$, number of blocks $L = 4$, number of heads $H = 4$, and batch size one remain fixed. All model measurements use CUDA AMP, 10 warm-up iterations, and 30 measured iterations with CUDA-event timing. Each model, node count, and execution phase is evaluated in an isolated process.

RAPTOR receives precomputed deterministic neighbor indices during the forward and training-step measurements, so graph construction is excluded from model latency and GPU-memory statistics. To expose that omitted cost, the exact three-dimensional graph builder based on CPU `torch.cdist` followed by `topk` is profiled separately using one warm-up run, ten measured runs at each node count, and eight CPU threads. Model-latency statistics are computed from 30 timed iterations, whereas the K-NN statistics are computed from 10 timed runs. Reported error bars represent one sample standard deviation. Peak memory denotes the maximum CUDA allocation observed during the measured phase.

Table 2. Common training configuration for the formal experiments.

Setting	Value
Training seeds	42 (all tasks); 41, 43 (AirfRANS comparison)
AirfRANS data-selection seed	2026
Nodes per sample	4096
Hidden dimension	64
Number of operator blocks	4
Number of attention heads	4
Physics slices or latent states	64
FFN expansion ratio	4
Local MLP hidden width	64
Slice-temperature initialization	0.5
Slice-temperature bounds	$[0.1, 5.0]$
RAPTOR neighborhood size K	12
RAPTOR gate initialization	$\alpha_l = 0$
Optimizer	AdamW
Initial learning rate	1×10^{-3}
Weight decay	1×10^{-6}
Maximum epochs	100
Early-stopping patience	15 epochs
Gradient clipping	1.0
Effective batch size	1 sample
Mixed precision	CUDA AMP; unsafe baseline kernels forced to FP32

5.4. Evaluation metrics

For AirfRANS, prediction accuracy is measured using one dataset-level relative L_2 error for each physical variable. For channel c, predictions and targets are concatenated over the M test cases (each containing N sampled nodes), giving

$$\varepsilon_c = \frac{\sqrt{\sum_{m=1}^{M} \left\| \widehat{\mathbf{Y}}_c^{(m)} - \mathbf{Y}_c^{(m)} \right\|_2^2}}{\sqrt{\sum_{m=1}^{M} \left\| \mathbf{Y}_c^{(m)} \right\|_2^2} + 10^{-12}}, \qquad c \in \{u_x, u_y, p/\rho, \nu_t\}. \tag{26}$$

The four-variable macro error is defined as

$$\varepsilon_{\text{macro}} = \frac{1}{4} \left(\varepsilon_{u_x} + \varepsilon_{u_y} + \varepsilon_{p/\rho} + \varepsilon_{\nu_t} \right). \tag{27}$$

Reported implementation measurements comprise trainable parameters, elapsed training time, and peak allocated GPU memory. End-to-end inference latency was not retained consistently for the accuracy experiments and is therefore not compared across trained runs. The separate node-scaling experiment instead reports isolated forward-pass and complete AdamW training-step latency, excluding data loading, post-processing, and K-NN graph construction.

For the automobile and aircraft experiments, the principal metric is ordinary MSE after standardizing C_p with the training-set mean and standard deviation. To distinguish it from the conventional ratio $\text{MSE}/\|Y\|_2^2$, we call it standardized-space MSE (sMSE):

$$\mathrm{sMSE} = \frac{1}{|D_{\mathrm{te}}|} \sum_{m \in D_{\mathrm{te}}} \frac{1}{N} \left\| \hat{\mathbf{Y}}^{(m)}_{\mathrm{norm}} - \mathbf{Y}^{(m)}_{\mathrm{norm}} \right\|_F^2 . \tag{28}$$

The best validation sMSE, training time, peak allocated GPU memory, and parameter count are recorded together with the test result. Per-object errors are additionally calculated to determine whether the overall improvement is consistently distributed across geometries rather than being dominated by a small number of test samples.

6. Experimental Results and Analysis

This section evaluates RAPTOR from five complementary perspectives: two-dimensional aerodynamic extrapolation, the contribution of the local adapter and its initialization, three-dimensional automobile C_p reconstruction, evaluation on aircraft geometries, and empirical node scaling, and the parameter-matched capacity and neighborhood-size studies. The detailed comparisons, ablations, capacity controls, and three-dimensional results use the reference training seed 42; the AirfRANS model comparison additionally reports training seeds 41 and 43. The complete code, configurations, split manifests, checkpoints, per-sample predictions, and evaluation scripts are available through the GitHub repository listed on the title page.

6.1. AirfRANS extrapolation results

6.1.1. Overall comparison

Tables 3 and 4 report the seed-42 relative L_2 errors on the Reynolds-number and angle-of-attack extrapolation tasks, respectively. Table 5 summarizes the corresponding macro errors over training seeds 41, 42, and 43. The macro error is the unweighted mean of the errors for u_x, u_y, p/ρ, and ν_t.

In the seed-42 Reynolds-number comparison, RAPTOR has the lowest macro relative L_2 error (0.2219), improving on the nearest baseline, Transolver-Tiny (0.2280), by 2.65% using the unrounded values. It is best for u_x and ν_t, whereas Transolver-Tiny and MoNo-Tiny lead for u_y and p/ρ, respectively. The macro advantage therefore reflects balanced errors rather than dominance on every output.

Table 3. Relative L_2 errors on the AirfRANS Reynolds-number extrapolation task with seed 42. The upper and middle groups contain conventional and modern baselines, respectively. The last column reports the relative macro-error reduction achieved by RAPTOR over each baseline. The best and second-best results are shown in bold and underlined, respectively. Displayed errors are rounded to four decimals; percentages are computed from unrounded values.

Method	u_x	u_y	p/ρ	ν_t	Macro	Relative improvement
MLP	0.2713	0.3892	0.5578	0.5127	0.4327	48.72%
GraphSAGE	0.2319	0.4269	0.5654	0.5185	0.4357	49.06%
PointNet	0.2050	0.2877	0.2338	0.3173	0.2610	14.96%
Graph U-Net	0.2255	0.2784	0.3022	0.4797	0.3214	30.96%
Sparse Token	0.2412	0.3309	0.4034	0.5043	0.3699	40.01%
Transolver-Tiny	<u>0.1516</u>	**0.2233**	0.2366	<u>0.3003</u>	<u>0.2280</u>	2.65%
Transolver++-Tiny	0.1684	0.2373	0.3434	0.3517	0.2752	19.36%
MoNo-Tiny	0.2825	0.5035	**0.2150**	0.3612	0.3406	34.84%
LinearNO-Tiny	0.1565	<u>0.2249</u>	0.2360	0.5360	0.2884	23.04%
RAPTOR (ours)	**0.1395**	0.2292	<u>0.2251</u>	**0.2939**	**0.2219**	–

Note: Relative improvement is computed as $\left(E_{\mathrm{baseline}} - E_{\mathrm{RAPTOR}}\right) / E_{\mathrm{baseline}} \times 100\%$, where E denotes the macro relative L_2 error.

In the seed-42 AoA comparison, RAPTOR again obtains the lowest macro error, 0.2969, improving upon Transolver-Tiny by 5.37%.

However, the advantage over LinearNO-Tiny is only 0.37%. The two methods should therefore be regarded as closely matched in this run. LinearNO-Tiny obtains the lowest u_x error, whereas RAPTOR performs best for u_y.

PointNet remains strongest for p/ρ and turbulent viscosity. RAPTOR achieves the lowest macro error because it avoids a large error in any one variable, rather than because it is uniformly best on all four channels.

These seed-42 results show that the macro advantage does not require RAPTOR to lead on every physical variable. The three-seed summary below tests how the overall ranking changes with the training initialization.

Table 4. Relative L_2 errors on the AirfRANS angle-of-attack (AoA) extrapolation task with seed 42. Displayed errors are rounded to four decimals; percentages are computed from unrounded values.

Method	u_x	u_y	p/ρ	ν_t	Macro	Relative improvement
MLP	0.3183	0.4498	0.6697	0.5652	0.5008	40.71%
GraphSAGE	0.2365	0.4086	0.5534	0.5420	0.4351	31.76%
PointNet	0.2433	0.3254	**0.3257**	**0.4357**	0.3326	10.71%
Graph U-Net	0.2224	0.3105	0.4030	0.5209	0.3642	18.47%
Sparse Token	0.2993	0.3267	0.4576	0.5982	0.4204	29.38%
Transolver-Tiny	0.1698	0.2446	0.3731	0.4676	0.3138	5.37%
Transolver++-Tiny	0.2317	0.3340	0.4456	0.5049	0.3790	21.67%
MoNo-Tiny	0.3286	0.5273	0.3542	0.6127	0.4557	34.84%
LinearNO-Tiny	**0.1416**	0.2272	0.3512	0.4720	0.2980	0.37%
RAPTOR (ours)	0.1510	**0.2189**	0.3350	0.4828	**0.2969**	–

Table 5. AirfRANS macro relative L_2 errors over training seeds 41, 42, and 43 (mean $\pm$ sample standard deviation; lower is better). Each macro error is the unweighted mean of the four variable-wise errors. All runs within a task use the same fixed data cache and 4096 sampled nodes.

Method	Reynolds number	AoA
MLP	0.4327 ± 0.0045	0.4858 ± 0.0233
GraphSAGE	0.4232 ± 0.0135	0.4463 ± 0.0163
PointNet	0.2634 ± 0.0074	0.3294 ± 0.0147
Graph U-Net	0.3178 ± 0.0086	0.3742 ± 0.0125
Sparse Token	0.3678 ± 0.0089	0.4156 ± 0.0224
Transolver-Tiny	0.2595 ± 0.0617	0.3202 ± 0.0119
Transolver++-Tiny	0.2916 ± 0.0228	0.3690 ± 0.0236
MoNo-Tiny	0.3391 ± 0.0044	0.4285 ± 0.0351
LinearNO-Tiny	0.2499 ± 0.0363	0.3132 ± 0.0277
RAPTOR (ours)	$\mathbf{0.2210 \pm 0.0019}$	$\mathbf{0.3083 \pm 0.0108}$

Across three seeds, RAPTOR has the lowest mean macro error among ten models on both tasks, but wins only two of three runs per task. LinearNO-Tiny is the closest baseline by mean on Reynolds number (0.2499 ± 0.0363) and AoA (0.3132 ± 0.0277); the AoA gap is not a consistent per-seed advantage.

Using the same seed label and data cache, but not shared initial weights, we define paired improvement from unrounded macro errors as $\Delta_{b,s} = 100(E_{b,s} - E_{\mathrm{RAPTOR},s})/E_{b,s}$. Against Transolver-Tiny, RAPTOR improves in all three common-seed comparisons on each task. For seeds 41, 42, and 43, the improvements are 32.77%, 2.65%, and 0.52% on Reynolds number and 1.02%, 5.37%, and 4.65% on AoA, with respective paired means of $(11.98 \pm 18.03)\%$ and $(3.68 \pm 2.33)\%$. The Reynolds result is sensitive to the seed-41 Transolver-Tiny run (macro 0.3305; ν_t error 0.6334); the other two margins are much smaller. Against LinearNO-Tiny, paired mean improvements are $(10.41 \pm 12.14)\%$ and $(1.22 \pm 6.16)\%$, respectively, with RAPTOR losing one seed on each task.

The three-seed variable-wise leaders differ from the seed-42 results. On Reynolds number, LinearNO-Tiny leads for u_x and u_y, MoNo-Tiny for p/ρ, and RAPTOR for ν_t. On AoA, LinearNO-Tiny leads for u_x, RAPTOR for u_y, MoNo-Tiny for p/ρ, and PointNet for ν_t. Relative to Transolver-Tiny, RAPTOR lowers AoA p/ρ error in all three seeds but raises ν_t error in all three. The macro gain is therefore not uniform across variables. These three-seed sample

standard deviations describe variation on one fixed split; they do not establish statistical superiority or generalization to new splits.

6.1.2. *Contribution of the local adapter and gate initialization*

To separate the effects of local correction and gate initialization, the complete RAPTOR model is compared with two variants: RAPTOR without the local adapter and RAPTOR with the adapter gate initialized to $\alpha_l = 0.1$. The complete model uses the proposed zero initialization $\alpha_l = 0$, as reported in Table 6.

Table 6. Core ablation results on AirfRANS. Values are four-variable macro relative L_2 errors. Percentages reported in the text are computed from unrounded values. The no-adapter entries correspond to the Transolver-Tiny backbone reported in Tables 3 and 4.

Variant	Reynolds	AoA
Without local adapter	0.2280	0.31377
Nonzero initialization, $\alpha_l = 0.1$	0.24556	0.32218
RAPTOR, $\alpha_l = 0$	**0.22192**	**0.29692**

On Reynolds-number extrapolation, RAPTOR improves by 9.63% relative to the nonzero-initialization variant. The no-adapter entry from Table 3 is an independently trained backbone reference, not a seed-matched ablation; the AoA no-adapter entry is matched. On AoA, RAPTOR improves by 5.37% relative to this matched backbone and by 7.84% relative to nonzero initialization. Table 6 reports macro errors only; matched variable-wise ablation results are unavailable.

The comparison between zero and nonzero initialization supports the baseline-preserving role of the proposed gate. Starting from $\tanh(\alpha_l) = 0$ allows the model to begin from the unperturbed Physics-Attention function and to activate local corrections only when they are supported by the optimization signal. Initializing the adapter with a nonzero contribution degrades both extrapolation tasks. This result is consistent with the intended optimization mechanism, although it does not by itself constitute a general convergence guarantee.

For Reynolds extrapolation, the four learned values of $\tanh(\alpha_l)$ are approximately -0.308, -0.168, 0.141, and 0.255. Their signs alone cannot be interpreted as reinforcement or suppression because $\gamma_l \mathbf{M}_l$ has a sign–scale non-identifiability: the local MLP can reverse or rescale its output. A more meaningful diagnostic is $\|\gamma_l \mathbf{M}_l\|_F / \|\mathbf{Z}_{\mathrm{PA}}^{(l)}\|_F$, which should be logged in future multi-seed runs.

6.2. Three-dimensional automobile C_p reconstruction

Table 7 presents the results for the 100-vehicle experiment. Transolver-Tiny is the matched no-adapter baseline.

Table 7. Results for the 100-vehicle surface-pressure-coefficient reconstruction experiment.

Model	Test sMSE	Parameters	Peak memory (MiB)	Training time (s)
PointNet	0.486309	21,057	23.95	2937.5
Transolver-Tiny	0.145534	195,345	103.76	2428.9
RAPTOR	**0.135684**	245,525	117.77	3317.2

RAPTOR reduces the overall test sMSE from 0.145534 for Transolver-Tiny to 0.135684, an aggregate relative reduction of 6.77%. Compared with PointNet, the reduction is 72.10%. These results suggest that global token interaction is important for the three-dimensional variable-geometry problem and that local adaptation provides a further improvement over the global backbone.

The per-object comparison shows that RAPTOR outperforms Transolver-Tiny on seven of the ten test vehicles. The mean paired improvement is 6.16%, the median paired improvement is 4.81%, and the bootstrap 95% confidence interval is $[0.61, 12.43]$%. The positive interval describes resampling variation among these ten fixed test objects only; it does not include training-seed uncertainty and should not be read as a population-level significance claim.

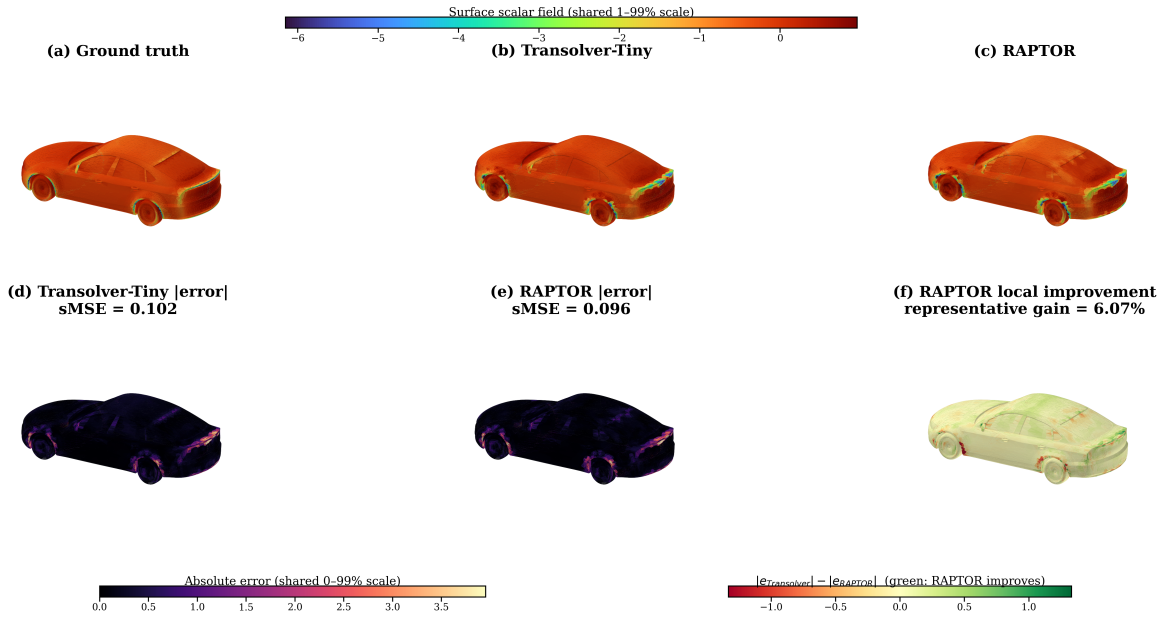


Figure 2: Automotive C_p reconstruction and spatial error reduction of RAPTOR relative to Transolver-Tiny.

Figure 2 compares the full-mesh C_p reconstruction and spatial error distributions of Transolver-Tiny and RAPTOR on a representative test vehicle. The sample is selected according to the median vehicle-level improvement of RAPTOR, rather than the maximum-gain case. The ground truth and two predictions in Figs. 2(a)–(c) share the same 1–99% field scale. Both models recover the dominant C_p distribution, while RAPTOR more closely reproduces the localized variations around the wheels, lower body, and rear geometric edges.

Figures 2(d) and 2(e) show the absolute errors of Transolver-Tiny and RAPTOR under a shared error scale. Their sMSEs are 0.10201 and 0.09582, respectively, corresponding to a 6.07% reduction for RAPTOR. The improvement map in Fig. 2(f) is defined as $\Delta e(\mathbf{x}) = \left|y(\mathbf{x}) - \hat{y}_{\text{Transolver}}(\mathbf{x})\right| - \left|y(\mathbf{x}) - \hat{y}_{\text{RAPTOR}}(\mathbf{x})\right|$.

Positive green regions indicate lower errors for RAPTOR, whereas negative red regions indicate local degradation. The distributed green regions suggest that the improvement is associated with neighborhood-scale corrections over several parts of the vehicle surface, particularly near geometric edges and wheel-related structures. Small red regions also show that the local adapter does not improve every surface location.

For visualization, predictions at the original 4,096 sampled nodes are mapped to the full triangular mesh by nearest-neighbor interpolation. All reported sMSE values are computed on the original sampled nodes and are therefore unaffected by this visualization-only interpolation.

To determine where the adapter is beneficial, each vehicle is partitioned using its own coordinate percentiles. Front and rear contain nodes below the 20th and above the 80th percentile of x, respectively. Within the middle 60%, roof contains nodes at or above the 70th percentile of z, lower/wheel-zone contains nodes at or below the 25th percentile, and side is the remainder. A regional sMSE is first averaged over nodes within each vehicle; Figure 3 reports the unweighted mean of the ten vehicle-level regional errors.

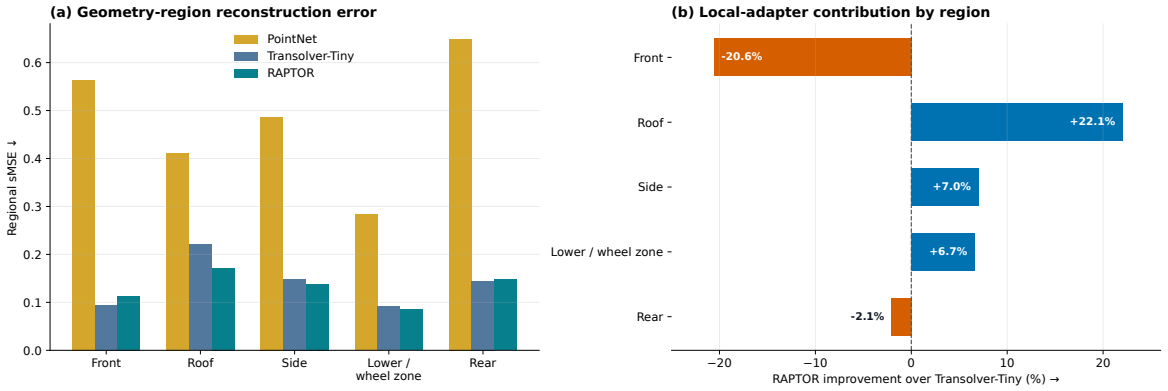


Figure 3: Region-wise reconstruction errors and local-adapter contributions on the ten-vehicle test set from the 100-vehicle experiment. (a) Mean sMSEs of PointNet, Transolver-Tiny, and RAPTOR over five vehicle-relative surface regions. (b) Relative regional improvement of RAPTOR over Transolver-Tiny. Positive values indicate lower errors for RAPTOR, whereas negative values indicate regional degradation. The local adapter provides the largest improvement over the roof, followed by the side and lower/wheel zone, but does not improve the front and rear regions.

Relative to Transolver-Tiny, RAPTOR lowers the mean regional error by 22.1% on the roof, 7.0% on the side, and 6.7% in the lower/wheel region. In contrast, its errors increase by 20.6% at the front and 2.1% at the rear. The local adapter is therefore most effective over the roof, side, and lower-body regions in the current experiment, but it does not uniformly improve every geometric region.

This observation is consistent with the per-vehicle comparison. The largest individual gain is 28.64%, whereas the identified regression case exhibits a 9.51% increase in error. The overall improvement consequently reflects a majority trend rather than universal dominance on every geometry.

RAPTOR increases the parameter count by 25.7% and peak allocated memory by 13.5%. The elapsed times in Table 7 come from independent single runs and are reported descriptively; without repeated timing measurements and variability estimates, they do not isolate architecture-only overhead.

6.3. Evaluation on three-dimensional aircraft geometries

The aircraft experiment evaluates the same RAPTOR architecture on four three-dimensional aircraft families. Both models are trained from scratch on the aircraft split. Table 8 reports the results.

RAPTOR reduces the overall test sMSE by 8.57% and the best validation sMSE by 10.45% relative to Transolver-Tiny. It also completes early stopping after 91 epochs, compared with 96 epochs for the baseline. The total training time nevertheless increases by approximately 7.5%, reflecting the additional cost of neighborhood aggregation.

Table 8
Overall results for the 100-aircraft geometry experiment.

Model	Test sMSE	Best val. sMSE	Parameters	Memory (MiB)	Time (s)
Transolver-Tiny	0.843596	0.5532	195,345	103.76	1795.3
RAPTOR	**0.771331**	**0.4954**	245,525	117.77	1929.6

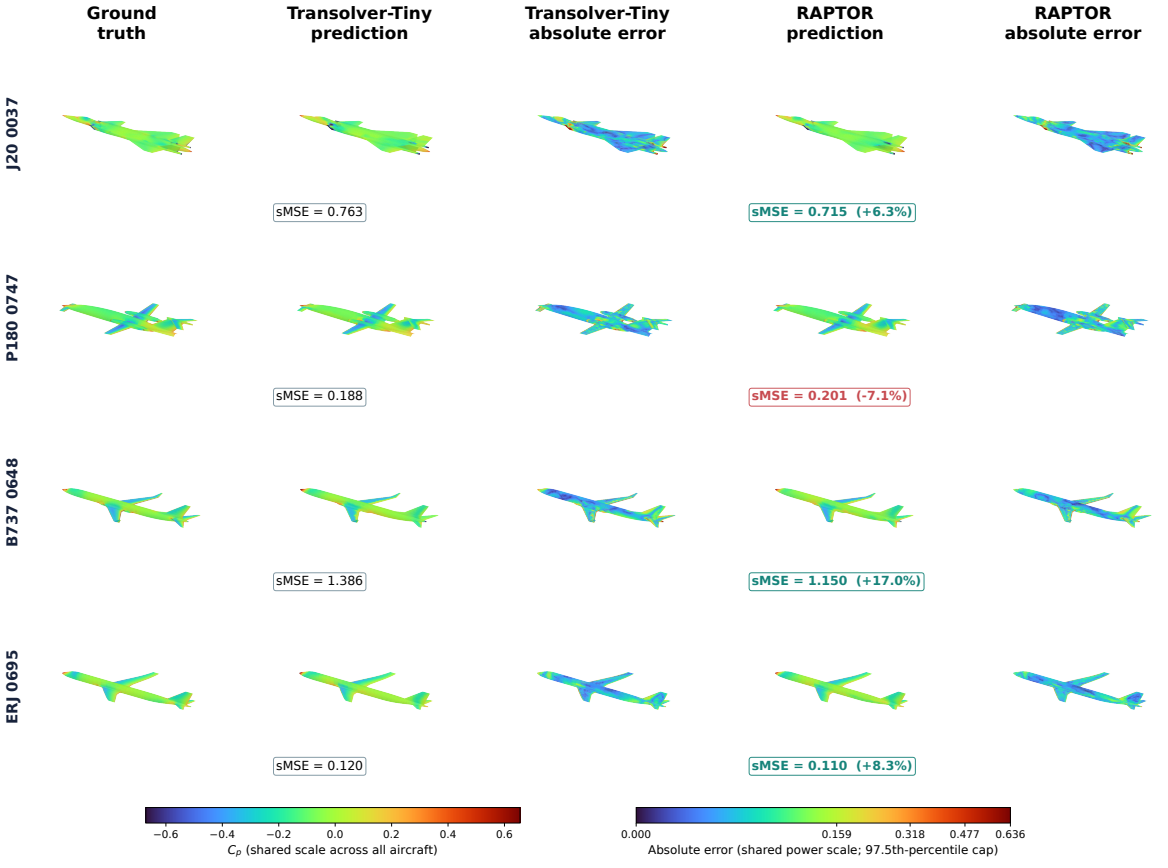


Figure 4: Representative aircraft C_p reconstruction for J20 sample 0037, P180 sample 0747, Boeing 737 sample 0648, and ERJ sample 0695. Columns show ground truth, Transolver-Tiny prediction and absolute error, and RAPTOR prediction and absolute error. Predictions and ground truth share a common field scale across the figure; all error maps use a common power-normalized error scale.

Figure 4 compares representative samples from the four aircraft families. RAPTOR reproduces the principal spatial variation of C_p across configurations with substantially different wings, fuselages, and tail structures. The residual-error maps are spatially nonuniform and tend to concentrate near geometrically intricate or rapidly varying regions.

The family-level statistics in Table 9 and Figure 4 reveal that the overall 8.57% improvement is not distributed equally among the four aircraft types.

RAPTOR improves the mean error for J20, Boeing 737, and ERJ, but performs 8.46% worse for P180. It wins on six of the ten individual aircraft and loses on four. This fixed-split result shows that RAPTOR can be applied to aircraft surface geometries without changing its principal architecture. At the same time, it identifies an important applicability boundary: a fixed 12-nearest-neighbor correction is not equally beneficial for every tested geometry. With only two P180 test objects, the result cannot be attributed to the family as a whole; it instead motivates further investigation of adaptive neighborhood scale, sampling density, and gate behavior.

Table 9. Aircraft-family test sMSE and RAPTOR improvement over Transolver-Tiny.

Family	Test samples	Transolver	RAPTOR	Improvement
J20	2	0.687817	**0.661924**	+3.76%
P180	2	**0.182439**	0.197882	−8.46%
Boeing 737	3	2.114592	**1.888884**	+10.67%
ERJ	3	0.117223	**0.109015**	+7.00%

Figure 5 presents the best paired case selected from the complete held-out aircraft test set according to the relative sMSE improvement of RAPTOR over Transolver-Tiny. For Boeing 737 sample 0483, RAPTOR reduces the sMSE from 3.26323 to 2.69088, corresponding to an improvement of 17.54%. Compared with the Transolver-Tiny error map, the RAPTOR result exhibits smaller deviations over several geometrically sensitive regions, particularly the wing surfaces, wing–body junctions, and tail assembly. This spatial pattern is consistent with the intended role of the local adapter in recovering neighborhood-scale information that may be weakened during physics-token compression.

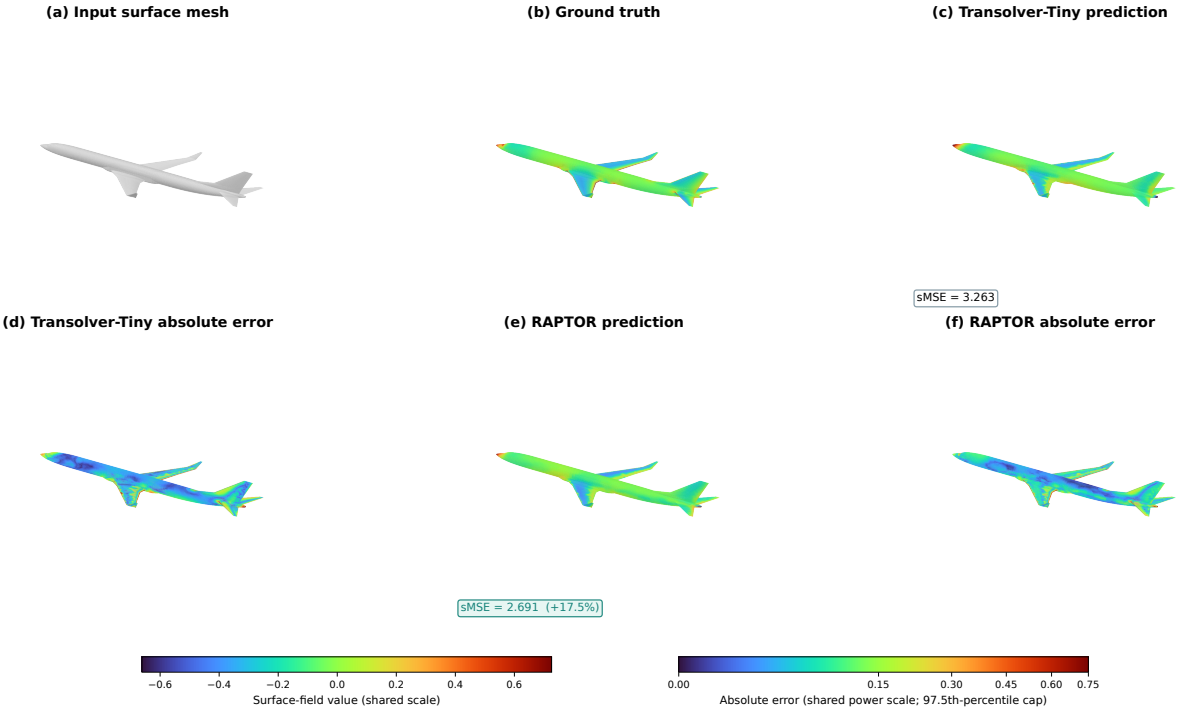


Figure 5: Best paired aircraft C_p-reconstruction case in the complete held-out test set. For Boeing 737 sample 0483, RAPTOR reduces the sMSE from 3.26323 to 2.69088, corresponding to a relative improvement of 17.54% over Transolver-Tiny. Panels (a)–(f) show the input surface mesh, ground truth, Transolver-Tiny prediction, Transolver-Tiny absolute error, RAPTOR prediction, and RAPTOR absolute error, respectively. The ground truth and predictions share a common field scale, while the two absolute-error maps share a common error scale.

Figure 6 presents the worst paired case in the same held-out test set. For P180 sample 1118, the sMSE increases from 0.17730 for Transolver-Tiny to 0.19492 for RAPTOR, corresponding to a relative improvement of −9.94%. Although both models preserve the overall C_p distribution, the RAPTOR error map indicates that the local correction does not reduce the remaining error for this geometry and slightly increases deviations over parts of the wing surfaces and component-junction regions.

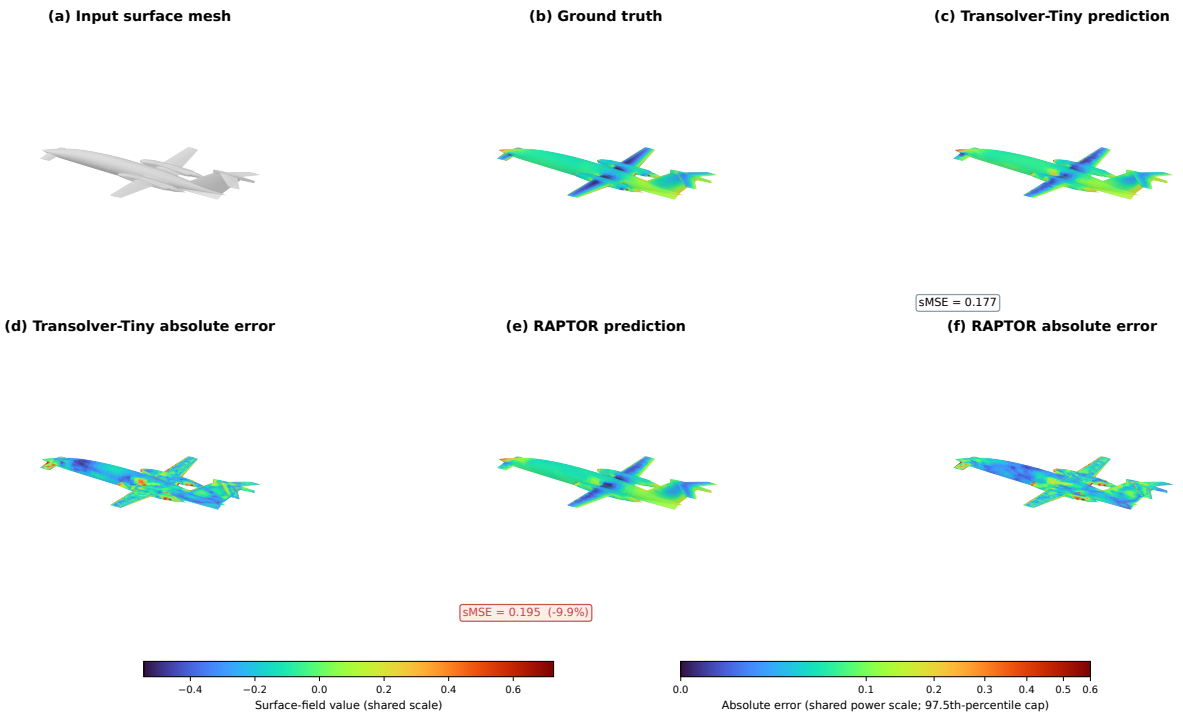


Figure 6: Worst paired aircraft C_p-reconstruction case in the complete held-out test set. For P180 sample 1118, the sMSE increases from 0.17730 for Transolver-Tiny to 0.19492 for RAPTOR, corresponding to a relative improvement of −9.94%. Panels (a)–(f) show the input surface mesh, ground truth, Transolver-Tiny prediction, Transolver-Tiny absolute error, RAPTOR prediction, and RAPTOR absolute error, respectively. The ground truth and predictions share a common field scale, while the two absolute-error maps share a common error scale.

Taken together, these two cases show that residual local adaptation is not uniformly beneficial for every aircraft geometry. RAPTOR's overall advantage should therefore be interpreted as an average improvement under the present fixed split and seed, rather than a universal per-sample gain. In both figures, the full-surface colors are used only for visualization and are obtained by interpolating the predictions from the matched 4,096 sampled nodes onto the original aircraft surface mesh. All reported sMSE values are computed directly on the matched sampled nodes rather than on the interpolated full-surface fields.

6.4. Empirical node-scaling and resource profile

Figure 7 and Table 10 report the isolated node-scaling profile under the fixed configuration $S = 64$, $K = 12$, $d = 64$, $L = 4$, and $H = 4$. Latency markers and error bars represent the mean and one sample standard deviation over 30 timed iterations for the model measurements. Transolver-Tiny and RAPTOR complete both forward and full mixed-precision training steps at $N = 16384$. At that resolution, their mean training-step latencies are 49.37 ± 2.33 ms and 66.99 ± 1.07 ms, respectively, while their peak allocated GPU memories are 350.70 MiB and 410.31 MiB. From $N = 4096$ to $N = 16384$, the corresponding memory footprints increase by factors of 3.41 and 3.50 for a fourfold increase in node count.

RAPTOR's mean training-step latency is not strictly monotonic at the intermediate node counts: it changes from 54.18 ± 6.27 ms at $N = 2048$ to 42.77 ± 8.09 ms at $N = 4096$ and 44.40 ± 4.15 ms at $N = 8192$. The overlapping one-standard-deviation ranges indicate that these intermediate differences should not be interpreted as an algorithmic speedup with increasing N. They are consistent with timing variability and changes in kernel-launch overhead and GPU utilization. The overall memory trend remains consistent with the linear-in-N activation structure expected when S, K, d, and L are fixed.

The explicit dense-attention baseline exhibits a different scaling regime. Its training-step allocation increases from 154.91 MiB at $N = 1024$ to 2098.40 MiB at $N = 4096$. At $N = 8192$, the training step completes only through Windows Display Driver Model shared-memory spillover: its peak CUDA allocation reaches 8274.59 MiB, exceeding the 6141 MiB physical device memory, and its mean latency reaches 12.51 ± 0.35 s.

Dense attention at $N = 16384$ was not attempted to avoid further memory oversubscription. Accordingly, the $N = 8192$ result should be interpreted as a spillover measurement rather than a conventional in-device-memory result.

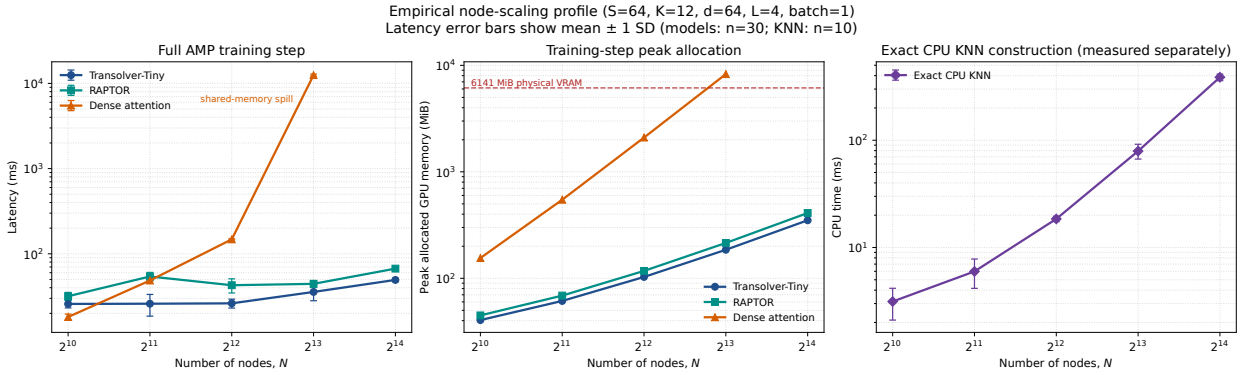


Figure 7: Empirical node-scaling profile on the RTX 4050 Laptop GPU. The left panel reports the mean latency of a complete CUDA-AMP AdamW training step, with error bars denoting one sample standard deviation over 30 timed iterations. The center panel reports peak allocated training-step GPU memory; error bars are not applicable because this quantity is the maximum allocation observed during each measured phase. The right panel reports the separately measured exact CPU K-NN construction time, with error bars denoting one sample standard deviation over 10 timed runs. Transolver-Tiny and RAPTOR use physics-token interaction with $S = 64$; RAPTOR additionally uses precomputed $K = 12$ neighbor indices. The dense baseline explicitly materializes per-head $N \times N$ attention scores. Its $N = 8192$ training step allocates 8274.59 MiB, exceeding the 6141 MiB physical GPU memory and invoking shared-memory spillover. Dense attention at $N = 16384$ was not attempted to avoid further memory oversubscription.

Table 10. Measured node-scaling results. Each model entry reports forward latency/training-step latency/peak allocated training-step memory in ms/ms/MiB. Model latencies are means over 30 timed iterations, whereas memory values are peak allocations. The K-NN column reports mean construction time ± one sample standard deviation over 10 timed runs. The final column gives the memory required to materialize the FP32 pairwise-distance matrix.

N	Transolver-Tiny	RAPTOR	Dense attention	KNN construction (mean ± SD, ms)	KNN distance matrix (MiB)
1024	6.34/25.80/40.50	9.25/31.76/44.82	5.10/18.13/154.91	3.14 ± 1.03	4
2048	6.30/25.95/61.27	14.31/54.18/68.79	19.08/48.58/546.74	5.98 ± 1.82	16
4096	6.58/26.21/102.94	14.27/42.77/117.33	53.07/147.83/2098.40	18.47 ± 1.04	64
8192	8.91/35.67/185.29	13.53/44.40/214.75	181.95/12512.48/8274.59[†]	79.21 ± 12.50	256
16384	14.31/49.37/350.70	21.74/66.99/410.31	–	386.52 ± 26.99	1024

The distance-matrix footprint is calculated as $4N^2/2^{20}$ MiB for an FP32 $N \times N$ matrix. [†]The dense-attention run completed using WDDM shared-memory spillover; its allocation exceeds the 6141 MiB physical GPU capacity and therefore is not an in-device-memory result. Dense attention at $N = 16{,}384$ was not attempted to avoid further memory oversubscription.

Exact Euclidean K-NN construction remains separate from the model measurements. Based on 10 timed runs at each node count, its mean CPU construction time increases from 3.14 ± 1.03 ms at $N = 1024$ to 386.52 ± 26.99 ms at $N = 16384$. Meanwhile, the materialized FP32 pairwise-distance matrix grows from 4 MiB to 1024 MiB. Its memory footprint increases by a factor of four whenever N doubles, directly reflecting the quadratic $\mathcal{O}(N^2)$ memory cost of the exact `torch.cdist`-based graph-construction procedure.

6.5. Parameter-matched capacity control and neighborhood-size sensitivity

To separate the effect of neighborhood information from that of increased model capacity, we evaluate a node-wise residual control. It retains the same bounded gate, LayerNorm, two-layer MLP, and parameter count as RAPTOR, but replaces the mean of neighboring features with the center-node feature. We also evaluate $K \in \{4, 8, 12, 24\}$ under the same AirfRANS split, sampling protocol, and training seed. The original $K = 12$ checkpoints are reused rather than retrained.

Table 11. Parameter-matched capacity control and K-sensitivity results on AirfRANS. Accuracy is four-variable macro relative L_2 error (lower is better). "Run peak" is the maximum allocated GPU memory recorded during training and final evaluation; "elapsed" is the duration of that complete run, including evaluation. R/A denotes Reynolds/AoA. The historical Transolver-Tiny resource values are omitted because they were not collected under the same run-level protocol. Each accuracy result uses one training seed.

Variant	Parameters	Reynolds	AoA	Run peak R/A (MiB)	Elapsed R/A (h)
Transolver-Tiny	195,796	0.22796	0.31377	–	–
Node-wise control	245,976	0.25663	0.31730	115.77/115.77	2.14/1.73
RAPTOR, $K = 4$	245,976	0.20475	0.31650	115.53/115.53	1.90/1.94
RAPTOR, $K = 8$	245,976	0.23277	0.30641	115.65/115.65	1.94/2.38
RAPTOR, $K = 12$	245,976	0.22192	0.29692	118.50/117.50	2.18/1.84
RAPTOR, $K = 24$	245,976	**0.20220**	**0.29494**	126.48/126.48	3.97/3.99

Table 11 shows that the parameter-matched node-wise control does not improve upon Transolver-Tiny on either task. Its macro error is 12.58% higher on Reynolds-number extrapolation and 1.12% higher on angle-of-attack (AoA) extrapolation. At the same parameter count, RAPTOR with $K = 12$ reduces error relative to this control by 13.53% and 6.42%, respectively. Under the present fixed split and seed, the extra parameters alone therefore do not account for RAPTOR's improvement; the comparison supports a contribution from local neighborhood information.

The response to neighborhood size is task-dependent. Reynolds-number extrapolation is non-monotonic in K: the $K = 8$ variant performs worse than Transolver-Tiny, whereas $K = 4$ and $K = 24$ obtain errors of 0.20475 and 0.20220. On AoA extrapolation, error decreases over the tested range as K increases. The $K = 24$ variant achieves the lowest error on both tasks, improving over the original $K = 12$ configuration by a further 8.89% on Reynolds and 0.67% on AoA. Thus, $K = 12$ is the preselected default, not the accuracy-optimal setting among those tested.

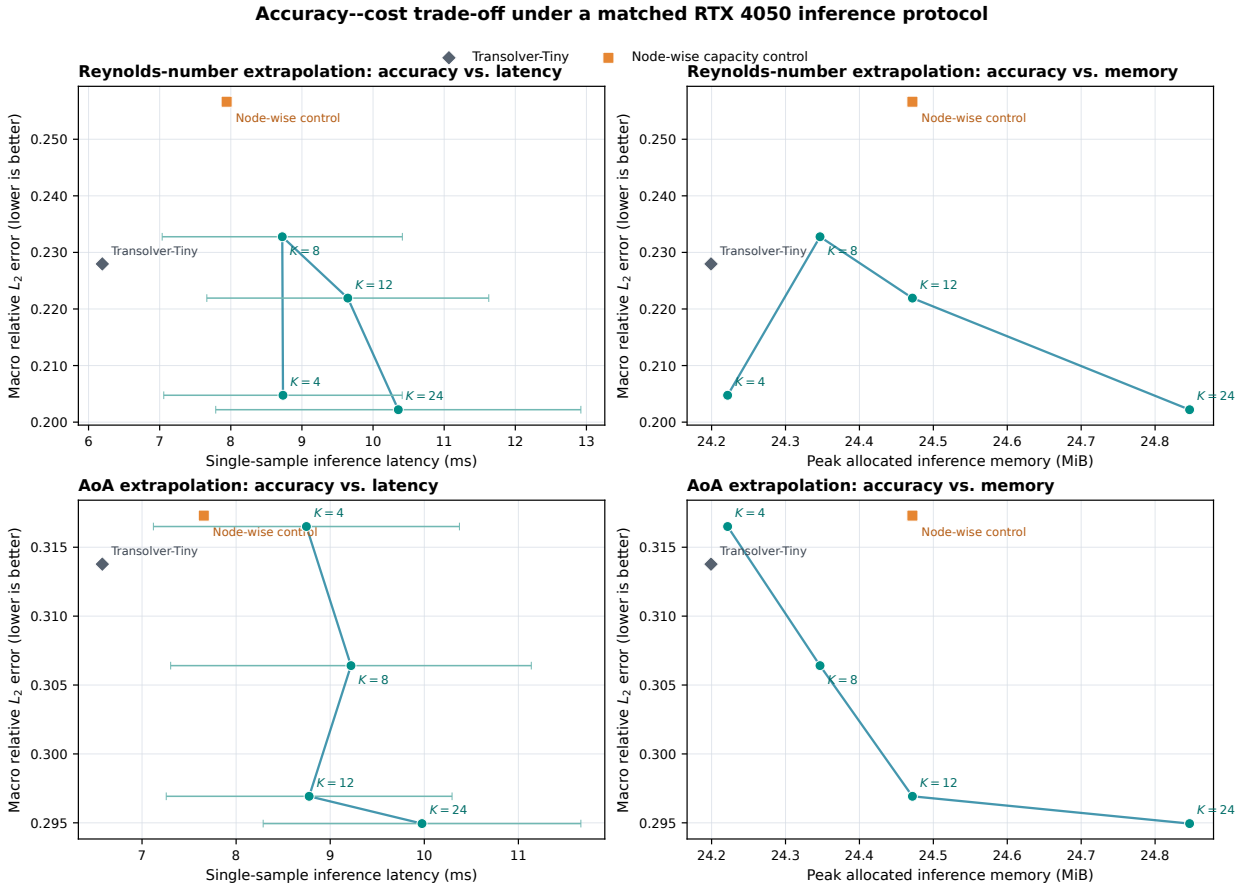


Figure 8: Accuracy–cost comparison at $N = 4096$ on an RTX 4050 Laptop GPU. Each latency point is the mean of 100 mixed-precision, single-sample forward passes after 20 warm-up passes; horizontal error bars represent one sample standard deviation. Memory points show peak allocated inference memory. Neighbor indices are precomputed, so K-NN graph construction is excluded from these inference measurements. Accuracy is obtained from one trained checkpoint per configuration and has no training-seed error bar.

As shown in Figure 8, increasing K from 12 to 24 raises the mean inference latency averaged over the two tasks by 10.37%, while peak allocated inference memory rises by 0.38 MiB. The run-level peak memory in Table 11 rises from 118.50/117.50 MiB to 126.48/126.48 MiB for Reynolds/AoA. The summed elapsed durations of the two training-and-evaluation runs are 4.02 h and 7.96 h for $K = 12$ and $K = 24$, respectively. These durations include final evaluation and come from separate single runs; they should not be interpreted as controlled estimates of architectural training overhead. We retain the preselected $K = 12$ setting for the main comparisons while explicitly reporting the accuracy advantage of $K = 24$.

Finally, the accuracy comparisons are descriptive results from one training seed and do not establish statistical significance across training runs. The 100 repeated forward passes characterize inference timing variability only, not variability in prediction accuracy.

7. Conclusions and Future Work

This study introduced RAPTOR, a neural operator that augments a Physics-Attention backbone with an explicit node-space K-NN residual adapter. The architecture combines token-mediated global communication with sparse local correction while retaining the computational advantages of physics-token compression. A bounded, zero-initialized gate ensures that each RAPTOR block initially reproduces the corresponding Physics-Attention backbone and allows the local contribution to be introduced progressively during optimization. Structurally, the resulting operator combines

a per-head conditional node-mixing pathway whose frozen rank is bounded by the number of physics slices with an $\mathcal{O}(NK)$ set of direct local dependencies. This characterization concerns the frozen node-mixing pattern and should not be interpreted as a rank bound on the complete nonlinear operator or its Jacobian.

Across three training seeds on the fixed AirfRANS splits, RAPTOR achieved the lowest mean macro relative L_2 error among ten evaluated models on both extrapolation tasks: 0.2210 ± 0.0019 for Reynolds number and 0.3083 ± 0.0108 for angle of attack. It reduced the error of its direct Transolver-Tiny backbone in all three same-seed comparisons on each task. Nevertheless, the Reynolds-number average is influenced by one high-error backbone run, and the AoA mean advantage over LinearNO-Tiny is small and not consistent across individual seeds. The seed-42 ablation results separately indicate that the local adapter improves the matched Physics-Attention backbone on AoA and that zero initialization is preferable to introducing a nonzero local correction at the start of training. These results support the intended baseline-preserving role of the gate, but neither the ablation nor the three-seed comparison establishes statistical superiority or a general optimization guarantee.

The three-dimensional experiments provide additional evidence that the global and local pathways are complementary. Relative to Transolver-Tiny, RAPTOR reduced the automobile test sMSE from 0.145534 to 0.135684, corresponding to a 6.77% improvement, and reduced the aircraft test sMSE by 8.57%. Nevertheless, the object-level results show that the benefit is not universal: RAPTOR improved seven of ten automobile objects and six of ten aircraft objects, while several individual geometries exhibited higher errors. The reported gains should therefore be interpreted as average improvements under the present protocols rather than guaranteed per-geometry behavior. Similarly, the measurements at $N = 4096$ demonstrate feasibility on the specified RTX 4050 laptop GPU. The isolated node-scaling profile further shows that Transolver-Tiny and RAPTOR complete mixed-precision training steps at $N = 16384$ with peak allocations of 350.70 MiB and 410.31 MiB, whereas explicit dense attention exceeds physical GPU memory at $N = 8192$. These measurements provide implementation-level evidence over the tested range, but they do not establish an accuracy–resource Pareto advantage or hardware-independent scalability. The seed-42 parameter-matched control indicates that extra adapter parameters alone do not account for the observed gains. The neighborhood-size study finds lower errors at $K = 24$ than at the preselected $K = 12$ for both AirfRANS tasks, but the summed durations of the two recorded training-and-evaluation runs rise from 4.02 to 7.96 h. The latter remains the uniform default under the present training budget, not an accuracy-optimal choice across the tested K values.

Future work should extend the three-seed AirfRANS comparison to additional training seeds and data splits, and repeat the single-seed three-dimensional experiments across seeds and larger object-level test sets. This would help separate training randomness from split-dependent variation. Resource evaluation should next vary S and explore a broader range of K, extend large-N measurements to additional hardware, and include end-to-end data movement, preprocessing, throughput, and accuracy–resource Pareto curves. The exact pairwise K-NN construction reaches 386.52 ± 26.99 ms over 10 timed runs and requires a 1024 MiB distance matrix at $N = 16384$, motivating cached, approximate, or geometry-aware graph builders for substantially larger meshes. The local connectivity also warrants further study because Euclidean neighbors can form nonphysical links across thin wings, vehicle gaps, or nearby but geodesically distant surfaces. Comparisons with mesh-edge, geodesic, radius-based, and adaptive neighborhoods may therefore improve both physical consistency and robustness. Finally, future analyses should quantify the realized contribution of the local branch through layer-wise norm ratios such as $\frac{\left\|\gamma_l \mathbf{M}_{\mathrm{loc}}^{(l)}\right\|_F}{\left\|\mathbf{Z}_{\mathrm{PA}}^{(l)}\right\|_F}$, rather than interpreting the sign or magnitude of the scalar gate alone. These extensions would clarify when explicit local adaptation is most useful and whether its benefits persist across resolutions, geometry families, and hardware regimes.

References

Adams, C., Ranade, R., Cherukuri, R., Choudhry, S., 2025. GeoTransolver: Learning Physics on Irregular Domains Using Multi-scale Geometry Aware Physics Attention Transformer. URL: https://arxiv.org/abs/2512.20399, doi:10.48550/arXiv.2512.20399, arXiv:2512.20399.

Azizzadenesheli, K., Kovachki, N., Li, Z., Liu-Schiaffini, M., Kossaifi, J., Anandkumar, A., 2024. Neural operators for accelerating scientific simulations and design. Nature Reviews Physics 6, 320–328. doi:10.1038/s42254-024-00712-5.

Bachlechner, T., Majumder, B.P., Mao, H.H., Cottrell, G.W., McAuley, J., 2021. ReZero is all you need: Fast convergence at large depth, in: Proceedings of the 37th Conference on Uncertainty in Artificial Intelligence, pp. 1352–1361. doi:10.48550/arXiv.2003.04887.

Bhatnagar, S., Afshar, Y., Pan, S., Duraisamy, K., Kaushik, S., 2019. Prediction of aerodynamic flow fields using convolutional neural networks. Computational Mechanics 64, 525–545. doi:10.1007/s00466-019-01740-0.

Bonnet, F., Mazari, J., Cinnella, P., Gallinari, P., 2022. AirfRANS: High-fidelity computational fluid dynamics dataset for approximating Reynolds-averaged Navier–Stokes solutions, in: Advances in Neural Information Processing Systems, pp. 23463–23478. doi:10.52202/068431-1705.

Brunton, S.L., Noack, B.R., Koumoutsakos, P., 2020. Machine learning for fluid mechanics. Annual Review of Fluid Mechanics 52, 477–508. doi:10.1146/annurev-fluid-010719-060214.

Chen, J., Xu, W., Xu, Z., Gutiérrez, N.G., Narra, S.P., McComb, C., 2025. Enforcing locality in neural operators for reducing generalization error. Journal of Computational Physics 538, 114131. doi:10.1016/j.jcp.2025.114131.

Chen, T., Chen, H., 1995. Universal approximation to nonlinear operators by neural networks with arbitrary activation functions and its application to dynamical systems. IEEE Transactions on Neural Networks 6, 911–917. doi:10.1109/72.392253.

Elrefaie, M., Morar, F., Dai, A., Ahmed, F., 2024. DrivAerNet++: A large-scale multimodal car dataset with computational fluid dynamics simulations and deep learning benchmarks, in: Advances in Neural Information Processing Systems, pp. 499–536. doi:10.52202/079017-0016.

Gao, H., Ji, S., 2019. Graph U-Nets, in: Proceedings of the 36th International Conference on Machine Learning, pp. 2083–2092. doi:10.48550/arXiv.1905.05178.

Hamilton, W.L., Ying, R., Leskovec, J., 2017. Inductive representation learning on large graphs, in: Advances in Neural Information Processing Systems, pp. 1024–1034. doi:10.48550/arXiv.1706.02216.

Hao, Z., Wang, Z., Su, H., Ying, C., Dong, Y., Liu, S., Cheng, Z., Song, J., Zhu, J., 2023. GNOT: A general neural operator transformer for operator learning, in: Proceedings of the 40th International Conference on Machine Learning, pp. 12556–12569. doi:10.48550/arXiv.2302.14376.

Hu, W., Liu, S., Qiao, P., Sun, Z., Dou, Y., 2026. Transolver is a linear transformer: Revisiting Physics-Attention through the lens of linear attention. Proceedings of the AAAI Conference on Artificial Intelligence 40, 408–416. doi:10.1609/aaai.v40i1.37003.

Immordino, G., Vaiuso, A., Ronch, A.D., Righi, M., 2025. Predicting transonic flowfields in non-homogeneous unstructured grids using autoencoder graph convolutional networks. Journal of Computational Physics 524, 113708. doi:10.1016/j.jcp.2024.113708.

Kissas, G., Seidman, J.H., Guilhoto, L.F., Preciado, V.M., Pappas, G.J., Perdikaris, P., 2022. Learning operators with coupled attention. Journal of Machine Learning Research 23, 1–63. doi:10.48550/arXiv.2203.07404.

Kontolati, K., Goswami, S., Karniadakis, G.E., Shields, M.D., 2024. Learning nonlinear operators in latent spaces for real-time predictions of complex dynamics in physical systems. Nature Communications 15, 5101. doi:10.1038/s41467-024-49411-w.

Kovachki, N., Li, Z., Liu, B., Azizzadenesheli, K., Bhattacharya, K., Stuart, A.M., Anandkumar, A., 2023. Neural operator: Learning maps between function spaces with applications to PDEs. Journal of Machine Learning Research 24, 1–97. doi:10.48550/arXiv.2108.08481.

Lee, H., Kim, J., Willette, J., Hwang, S.J., 2024. SEA: Sparse Linear Attention with Estimated Attention Mask, in: Proceedings of the 12th International Conference on Learning Representations. URL: https://openreview.net/forum?id=JbcwfmYrob, doi:10.48550/arXiv.2310.01777, arXiv:2310.01777.

Li, Z., Huang, D.Z., Liu, B., Anandkumar, A., 2023. Fourier neural operator with learned deformations for PDEs on general geometries. Journal of Machine Learning Research 24, 1–26. doi:10.48550/arXiv.2207.05209.

Li, Z., Kovachki, N., Azizzadenesheli, K., Liu, B., Bhattacharya, K., Stuart, A.M., Anandkumar, A., 2021. Fourier neural operator for parametric partial differential equations, in: International Conference on Learning Representations. doi:10.48550/arXiv.2010.08895.

Liu, Q., Zhong, W., Meidani, H., Abueidda, D., Koric, S., Geubelle, P.H., 2026. Geometry-informed neural operator transformer. Computer Methods in Applied Mechanics and Engineering 451, 118668. doi:10.1016/j.cma.2025.118668.

Lu, L., Jin, P., Pang, G., Zhang, Z., Karniadakis, G.E., 2021. Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators. Nature Machine Intelligence 3, 218–229. doi:10.1038/s42256-021-00302-5.

Luo, H., Wu, H., Zhou, H., Xing, L., Di, Y., Wang, J., Long, M., 2025. Transolver++: An accurate neural solver for PDEs on million-scale geometries, in: Proceedings of the 42nd International Conference on Machine Learning, pp. 41432–41449. doi:10.48550/arXiv.2502.02414.

Pfaff, T., Fortunato, M., Sanchez-Gonzalez, A., Battaglia, P.W., 2021. Learning mesh-based simulation with graph networks, in: International Conference on Learning Representations. doi:10.48550/arXiv.2010.03409.

Qi, C.R., Su, H., Mo, K., Guibas, L.J., 2017. PointNet: Deep learning on point sets for 3D classification and segmentation, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, pp. 652–660. doi:10.1109/CVPR.2017.16.

Thuerey, N., Weißenow, K., Prantl, L., Hu, X., 2020. Deep learning methods for Reynolds-averaged Navier–Stokes simulations of airfoil flows. AIAA Journal 58, 25–36. doi:10.2514/1.J058291.

Wen, G., Li, Z., Azizzadenesheli, K., Anandkumar, A., Benson, S.M., 2022. U-FNO—An enhanced Fourier neural operator-based deep-learning model for multiphase flow. Advances in Water Resources 163, 104180. doi:10.1016/j.advwatres.2022.104180.

Wen, S., Kumbhat, A., Lingsch, L.E., Mousavi, S., Zhao, Y., Chandrashekar, P., Mishra, S., 2025. Geometry aware operator transformer as an efficient and accurate neural surrogate for PDEs on arbitrary domains, in: Advances in Neural Information Processing Systems. doi:10.52202/085713-5197.

Wu, H., Luo, H., Wang, H., Wang, J., Long, M., 2024. Transolver: A fast transformer solver for PDEs on general geometries, in: Proceedings of the 41st International Conference on Machine Learning, pp. 53681–53705. doi:10.48550/arXiv.2402.02366.

Yang, Z., Wu, X., Fu, D., 2026. MoNo: Multiscale optimal transport neural operator for solving PDEs on general geometries. doi:10.48550/arXiv.2608.09764, arXiv:2608.09764. preprint.

Yin, M., Charon, N., Brody, R., Lu, L., Maggioni, M., Trayanova, N., 2024. A scalable framework for learning the geometry-dependent solution operators of partial differential equations. Nature Computational Science 4, 928–940. doi:10.1038/s43588-024-00732-2.

Zeng, C., Zhang, Y., Zhou, J., Wang, Y., Wang, Z., Liu, Y., Wu, L., Huang, D.Z., 2025. Point cloud neural operator for parameterized partial differential equations on irregular geometries. Computer Methods in Applied Mechanics and Engineering 443, 118022. doi:10.1016/j.cma.2025.118022.

Zhong, W., Liu, Q., Abueidda, D., Koric, S., Meidani, H., 2025. A comprehensive comparison of neural operators for 3D industry-scale engineering designs. URL: https://arxiv.org/abs/2510.05995, doi:10.48550/arXiv.2510.05995, arXiv:2510.05995.